

MACSE518 - Database Modelling and Design

QuakeWatch

A Real-Time Earthquake Data Platform Using ArangoDB and Django

Review-2 Document

DB Export, Screenshots of 500+ Records, Views, Triggers, Procedures and Indexes, 6_Queries (15 AQL Queries with Output), 7_TestCases, GitHub Link and CRUD Demonstration

Team details :

1 member : Arkkadu Bharath Reddy – 26MAI0058

Submitted by :

Arkkadu Bharath Reddy

26MAI0058

QuakeWatch: Performance and Consistency Evaluation of Revision-Aware Earthquake Data Management Using ArangoDB

Review 2 Report

Database Implementation, Query Evaluation, Integrity Mechanisms, Backend-Database Connectivity and Proposed Research Extension

Item	Detail
Subject	Database Management Systems (NoSQL database project)
Database	ArangoDB 3.12.5 (only database used)
Backend / Frontend	Python 3.12, Django, Django REST Framework / HTML, CSS, JavaScript, Bootstrap, Leaflet, Chart.js
Data source	U.S. Geological Survey (USGS) earthquake feeds, live ingestion
Repository	https://github.com/abharathreddy751/quake_watch_v2.git
Report date	11 October 2026

Safety statement: QuakeWatch does not predict earthquakes; its alerts are application notifications and not official emergency warnings. Earthquake data courtesy of the U.S. Geological Survey (USGS).

Contents

Abstract 5

Research Motivation and Scope 5

Mapping of the Review 2 Deliverables to the ArangoDB Stack 5

1. Environment and Architecture 6

1.1 Runtime environment 6

2. Database Implementation 7

2.1 Database and collection creation 7

2.2 Document design 9

2.3 Graph model 9

2.4 Sample data and record count 10

2.5 Indexes 11

2.6 Views 13

2.7 Triggers and procedures: NoSQL equivalents 14

2.8 Database export (dump) 14

3. Complex AQL Queries with Output 15

3.1 Query 1: Event count and magnitude statistics per region 16

3.2 Query 2: Dashboard summary with subqueries 17

3.3 Query 3: Per-network statistics joined to network names 18

3.4 Query 4: Revisions with large magnitude changes (array index) 19

3.5 Query 5: Ingestion latency percentiles 20

3.6 Query 6: Significant events per ISO week 21

3.7 Query 7: A user's alert rules with match counts (edge traversal) 22

3.8 Query 8: Matched events through a two-hop graph traversal 23

3.9 Query 9: Event relationships (OUTBOUND and INBOUND traversal) 24

3.10 Query 10: Map viewport search (GEO_CONTAINS) 25

3.11 Query 11: Radius search (GEO_DISTANCE) 26

3.12 Query 12: Location text search (ArangoSearch view) 27
3.13 Query 13: Ingestion run statistics by kind 28
3.14 Query 14: Depth histogram (bucketed aggregation) 29
3.15 Query 15: Nearby events in space and time 30
3.16 Summary of the query execution 30
4. Backend and Database Connectivity: CRUD Operations 31
4.1 Connectivity 31
4.2 CRUD operations demonstrated 31
5. Test Cases and Results 32
6. Proposed Research Extension 33
6.1 Evidence status 34
6.2 Index configurations 34
6.3 Workloads and datasets 34
6.4 Metrics and consistency check 35
6.5 Controls and limits 35
7. Conclusion and Future Work 35
References 36

Abstract

QuakeWatch ingests real earthquake observations from the USGS, validates and normalises them, keeps every revision of every event, and exposes search, map, analytics, alerting and research-snapshot functions through a Django REST API. ArangoDB is the only database and the only stateful service: it stores domain data, accounts and sessions, the durable job queue, caches, metrics and audit records. This report documents the implemented system and its verification: the database (2,670 live earthquake documents, 31 collections, 32 secondary indexes, an ArangoSearch view and a named graph), the mechanisms that play the role of triggers and stored procedures in a NoSQL setting, fifteen AQL queries with screenshot evidence, a verified CRUD demonstration, and the executed test cases. Building on this verified base, the report proposes a small research extension that evaluates ArangoDB performance and consistency under concurrent read/write workloads for three index configurations. The proposed experiments have not been executed yet and no benchmark result is claimed for them.

Research Motivation and Scope

Revision-aware earthquake data has an unusual access pattern: new and corrected observations arrive continuously, every change must append an immutable revision, and users read the same collections at the same time through search, map and analytics queries. Secondary indexes speed up reads but add work to every write, and concurrent writers to the same event can conflict. The research question of the extension is therefore workload-specific: how do index configurations trade read latency, write throughput, resource use and revision consistency inside ArangoDB under read-heavy, balanced and write-heavy loads. The study stays within the existing application and the existing database; it does not compare ArangoDB with other databases and does not claim superiority of any kind.

Mapping of the Review 2 Deliverables to the ArangoDB Stack

The Review 2 checklist is written with relational terminology. Because QuakeWatch uses a NoSQL multi-model database, each item is satisfied by the closest equivalent construct. Where ArangoDB has no direct counterpart (native triggers and stored procedures) this is stated explicitly and the mechanism that actually enforces the behaviour is demonstrated.

Table 1: Review 2 checklist mapped to the implemented NoSQL stack

Review 2 item	ArangoDB / QuakeWatch equivalent	Section
DB dump (.sql)	Native arangodump export (structure and data per collection)	2.8
Screenshots of 500+ records	events collection holds 2,670 real documents (2,655 at the time of Figure 6)	2.4
Views	ArangoSearch view events_search and the named graph quakewatch_graph	2.3, 2.6
Triggers	Strict JSON-schema validation, unique and TTL indexes, ingestion observers	2.7
Procedures	Atomic stream transactions and a catalogue of 53 named parameterised AQL queries	2.7
Indexes	32 named secondary indexes (persistent, geo, TTL) plus primary and edge indexes	2.5
15+ complex queries with output	15 AQL queries with GUI screenshots of query and output	3
Test cases	Executed test cases and the automated regression suite	5
GitHub link, CRUD working, backend and DB connectivity	Repository link, health check, CRUD table and demonstration video	4

1. Environment and Architecture

1.1 Runtime environment

The application runs locally with Docker Compose as four containers that share one image: the ArangoDB 3.12.5 server, the Django web process (gunicorn), a background worker and a scheduler. Jobs and schedules are documents in ArangoDB, claimed with atomic AQL updates, so no relational database, Redis or Celery is used. The ArangoDB web interface was accessed with the administrator account; the password is masked in Figure 1.

Table 2: Runtime components

Component	Technology	Role
Database	ArangoDB 3.12.5 (port 8529)	Only stateful service: events, revisions, users, sessions, jobs, cache, metrics, audit
Web	Django, Django REST Framework, gunicorn (port 8000)	Pages and versioned JSON API under /api/v1/
Worker	apps.jobs worker	Executes queued ingestion, enrichment and alert-evaluation jobs
Scheduler	apps.jobs scheduler	Enqueues the USGS feed synchronisation every 300 seconds
Driver	python-arango	Used only inside repository modules (enforced by architecture tests)

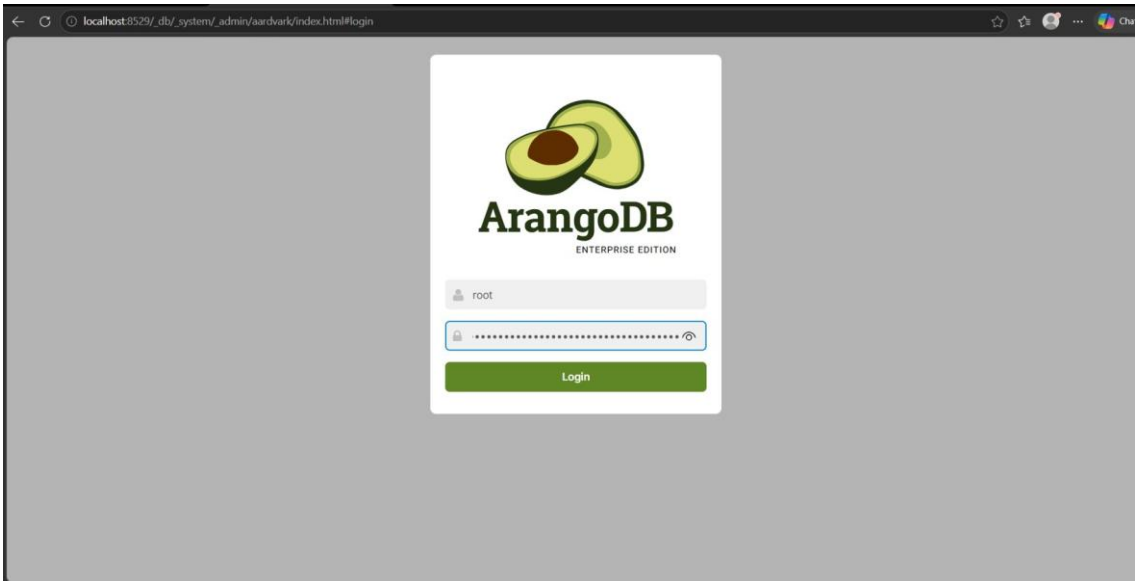


Figure 1: ArangoDB web interface login on localhost:8529 (password masked by the browser).

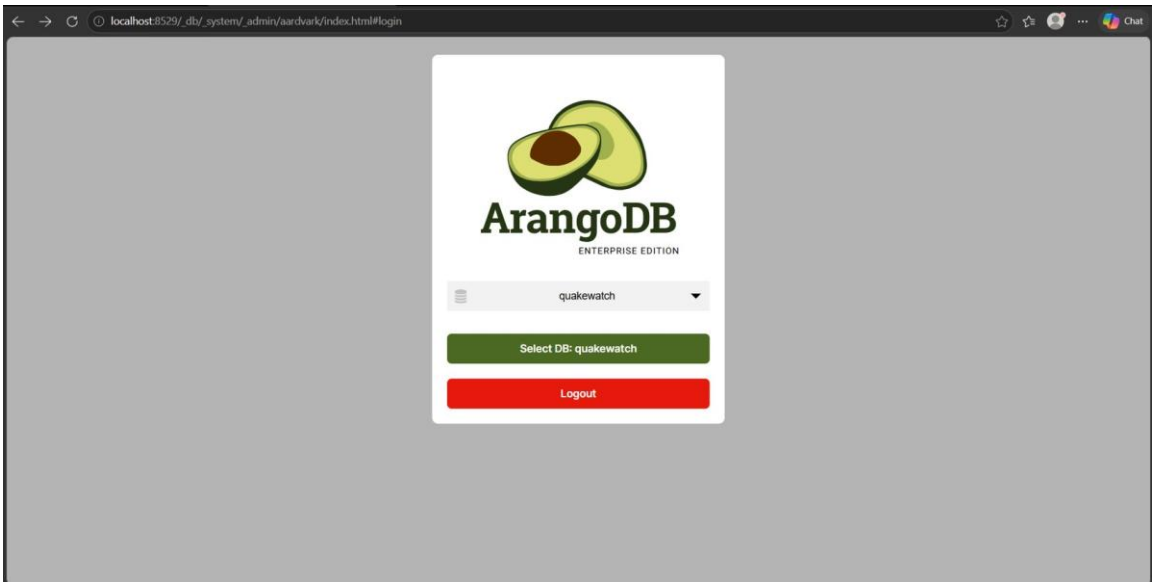


Figure 2: Database selection: the application database quakewatch is chosen after authentication.

2. Database Implementation

2.1 Database and collection creation

The database quakewatch and its structure are created by the idempotent management command arango_migrate (schema version 6): collections with JSON-schema validators, persistent, geo and TTL indexes, the ArangoSearch view and the named graph. Re-running it changes nothing, so the structure is reproducible. Figure 3 lists the 31 collections (27 document and 4 edge collections); Table 3 gives the purpose and live document count of each.



Figure 3: ArangoDB web interface listing the 31 QuakeWatch collections (document and edge types).

Table 3: Collections, live document counts and purpose (total 17,542 documents)

Collection	Type	Documents	Purpose
alert_matches	document	0	Notification records produced when a rule matches an event
alert_rules	document	1	User-defined alert rules (owner-scoped)
audit_log	document	22	Security and administrative audit trail
auth_users	document	2	Account records (password hashes, staff flags)
backfill_jobs	document	2	Historical backfill requests and progress
cache_entries	document	3	Query cache with TTL
circuit_breakers	document	0	Upstream circuit-breaker state
dead_letters	document	0	Records that failed validation, kept for retry
event_identities	document	3,402	Maps every USGS associated id to its canonical event (merge support)
event_in_region	edge	2,670	Edge: event to region
event_reported_by_network	edge	2,670	Edge: event to reporting network
event_revisions	document	2,767	Immutable per-change history of each event (before/after, changed fields)
events	document	2,670	Canonical earthquake documents, one per USGS event
id_sequences	document	1	Atomic integer sequences for ids
ingestion_runs	document	148	One document per feed sync, backfill or enrichment run
job_leases	document	0	Distributed leases with TTL
job_schedules	document	2	Periodic job definitions
jobs	document	98	Durable job queue
metric_counters	document	11	Operational counters
metric_series	document	77	Latency samples for percentile dashboards
networks	document	15	Reporting seismic networks (us, ak, nc, ...)
rate_limits	document	2	API throttle counters (TTL-expired)
raw_payloads	document	2,843	Original USGS payloads kept for provenance (TTL-expired)
regions	document	126	Normalised region dimension (US states, countries)
research_snapshots	document	0	Frozen, checksummed event selections for reproducible research
rule_matched_event	edge	0	Edge: rule to matched event
saved_searches	document	0	Per-user saved explorer filters (capped at 50)
schema_meta	document	1	Applied schema version marker
sessions	document	7	Django session store (TTL-expired)
user_owns_rule	edge	1	Edge: user to alert rule

Collection	Type	Documents	Purpose
users	document	1	Identity vertices for the graph

2.2 Document design

Documents are denormalised where reads dominate and referenced by key where relationships matter. An event is stored once under a deterministic key derived from the USGS identifier; its region and network are edges, and every change is an immutable document in event_revisions. Figure 6 shows real event documents in the Content tab. The events collection carries a strict JSON-schema validator that restricts magnitude, coordinates, status and timestamp format and rejects non-conforming writes on the server, whichever client sends them (Figure 4; behaviour demonstrated in Section 2.7).

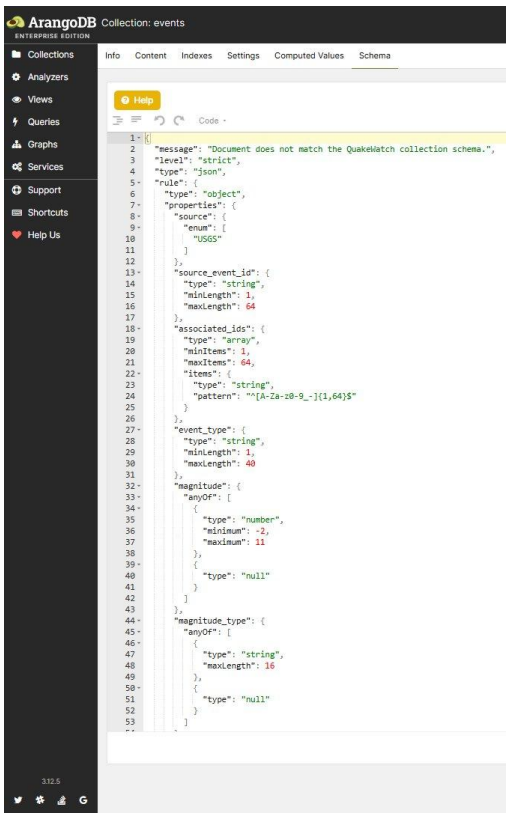


Figure 4: Schema tab of the events collection: server-side JSON-schema validation rules.

2.3 Graph model

The named graph quakewatch_graph connects users to alert rules, rules to matched events, events to regions and events to networks. Traversals answer which events matched a user's rules (Query 8) and which events share a region with a given event (Query 9).

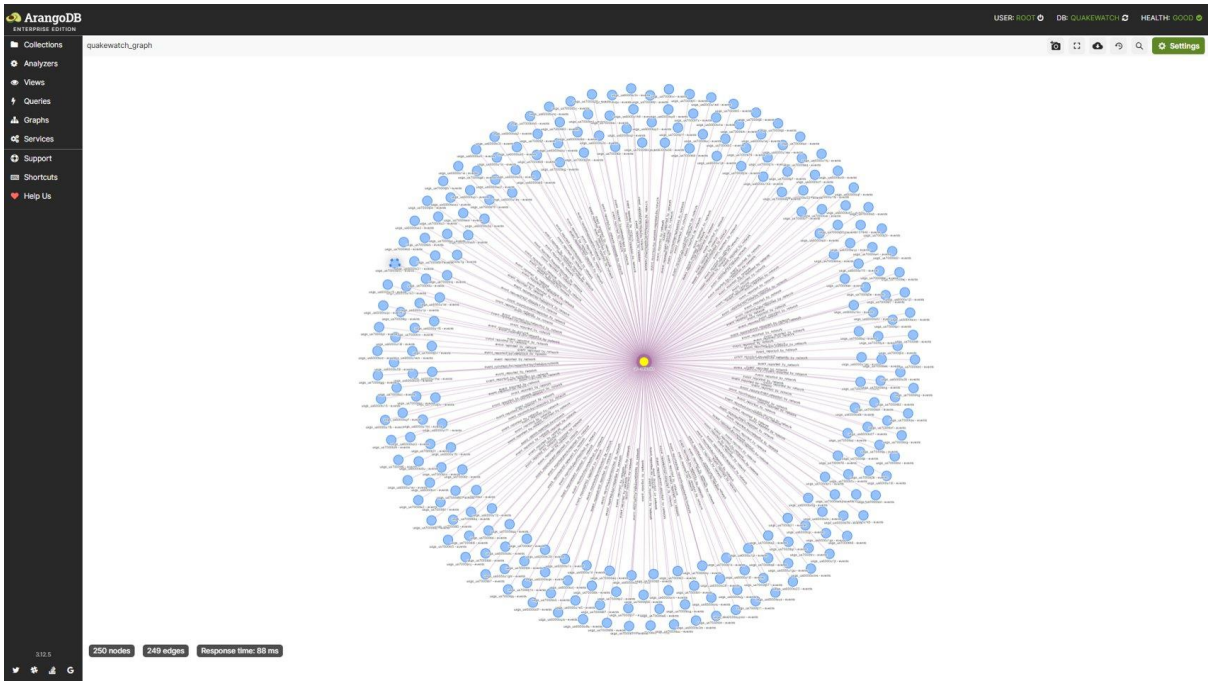


Figure 5: Graph viewer showing quakewatch_graph (250 nodes and 249 edges in the sampled view).

2.4 Sample data and record count

The data are real USGS observations. The events collection contains 2,670 documents from 2026-08-26 to 2026-10-11 with magnitudes from -1.05 to 6.6; the revision history holds 2,767 documents and the database 17,542 documents in total, which satisfies the requirement of at least 500 records. The count grows as the scheduler ingests the feed every five minutes, so the screenshot (2,655 documents) was taken slightly earlier than the tabulated figures.

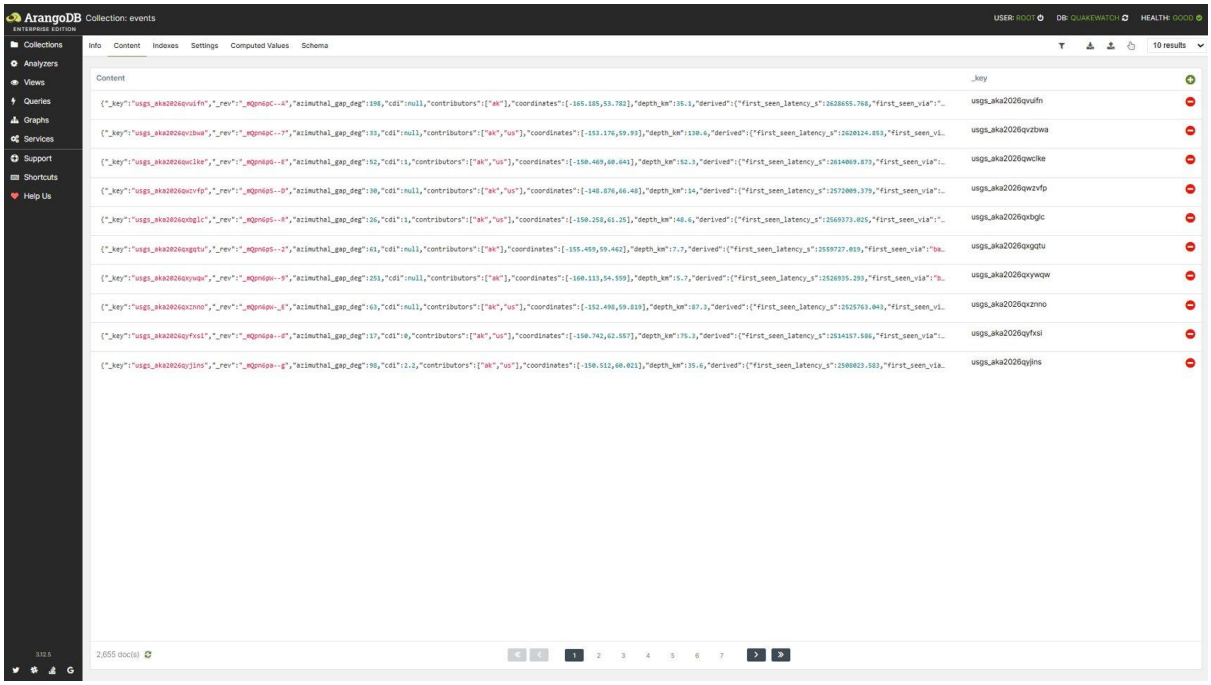


Figure 6: Content tab of the events collection showing 2,655 documents at the time of capture.

Table 4: Record counts of the principal entities (live database)

Entity	Count	Remark
events	2,670	Canonical earthquakes (2026-08-26 to 2026-10-11)
event_revisions	2,767	97 revisions beyond the initial creation records
event_identities	3,402	USGS associated ids mapped to canonical events
raw_payloads	2,843	Provenance copies of source payloads
event_in_region + event_reported_by_network	5,340	2,670 edges each
ingestion_runs	148	138 feed syncs, 8 enrichments, 6 backfill windows
regions / networks	126 / 15	Dimension collections

2.5 Indexes

Table 5: The 32 named secondary indexes defined by the schema

Index	Collection	Type	Fields	Unique	Purpose
qw_matches_rule_event	alert_matches	persistent	rule_key, event_key	yes	Unique match per (rule, event): no duplicate notifications
qw_matches_user_time	alert_matches	persistent	user_id, matched_at	no	Notification feed per user
qw_rules_user	alert_rules	persistent	user_id	no	Rules of one user
qw_rules_enabled	alert_rules	persistent	enabled	no	Select enabled rules during evaluation
qw_audit_at	audit_log	persistent	at	no	Audit trail by time
qw_auth_username	auth_users	persistent	username	yes	Unique username
qw_auth_username_ci	auth_users	persistent	username_ci	no	Case-insensitive username lookup
qw_backfill_status	backfill_jobs	persistent	status	no	Backfill jobs by status
qw_cache_ttl	cache_entries	ttl	expires_at	no	TTL: expire cache entries
qw_dead_last_failure	dead_letters	persistent	last_failure_at	no	Dead letters by last failure
qw_dead_source_event	dead_letters	persistent	source, source_event_id	no	Dead letters per source event

Index	Collection	Type	Fields	Unique	Purpose
qw_identities_event_key	event_identities	persistent	event_key	no	Resolve all ids of an event
qw_revisions_event_rev	event_revisions	persistent	event_key, revision_number	yes	Unique (event, revision number); history of one event
qw_revisions_observed_at	event_revisions	persistent	observed_at	no	Time-bounded revision analytics
qw_revisions_changed_fields	event_revisions	persistent	changed_fields[*]	no	Array index: find revisions that changed a given field
qw_events_geo	events	geo	coordinates	no	Radius and viewport searches (GEO_DISTANCE, GEO_CONTAINS)
qw_events_source_identity	events	persistent	source, source_event_id, merged_into	yes	Uniqueness of (source, source_event_id, merged_into); idempotent upsert
qw_events_occurred_at	events	persistent	occurred_at, _key	no	Time-ordered explorer pages and analytics windows
qw_events_magnitude	events	persistent	magnitude, _key	no	Magnitude-sorted listings and filters
qw_runs_started_at	ingestion_runs	persistent	started_at	no	Chronological run listing
qw_runs_status	ingestion_runs	persistent	status	no	Filter runs by status
qw_runs_kind_started	ingestion_runs	persistent	kind, started_at	no	Latest runs per kind (operations dashboard)
qw_leases_ttl	job_leases	ttl	expires_at	no	TTL: release abandoned leases
qw_jobs_claim	jobs	persistent	status, run_at	no	Claim the next runnable job
qw_jobs_dedupe	jobs	persistent	dedupe_key	yes	Unique dedupe key: no duplicate jobs
qw_jobs_task_created	jobs	persistent	task, created_at	no	Job history per task
qw_jobs_purge	jobs	ttl	purge_at	no	TTL: remove finished jobs
qw_rate_limits_ttl	rate_limits	ttl	expires_at	no	TTL: expire throttle counters
qw_raw_ttl	raw_payloads	ttl	expires_at	no	TTL: expire raw payload copies
qw_snapshots_user	research_snapshots	persistent	user_id, created_at	no	Snapshots per user
qw_saved_user	saved_searches	persistent	user_id, created_at	no	Saved searches per user
qw_sessions_ttl	sessions	ttl	expire_at	no	TTL: expire sessions

Figure 7 shows the events indexes in the interface. The index actually chosen by the optimizer for each of the fifteen queries is reported in Section 3.

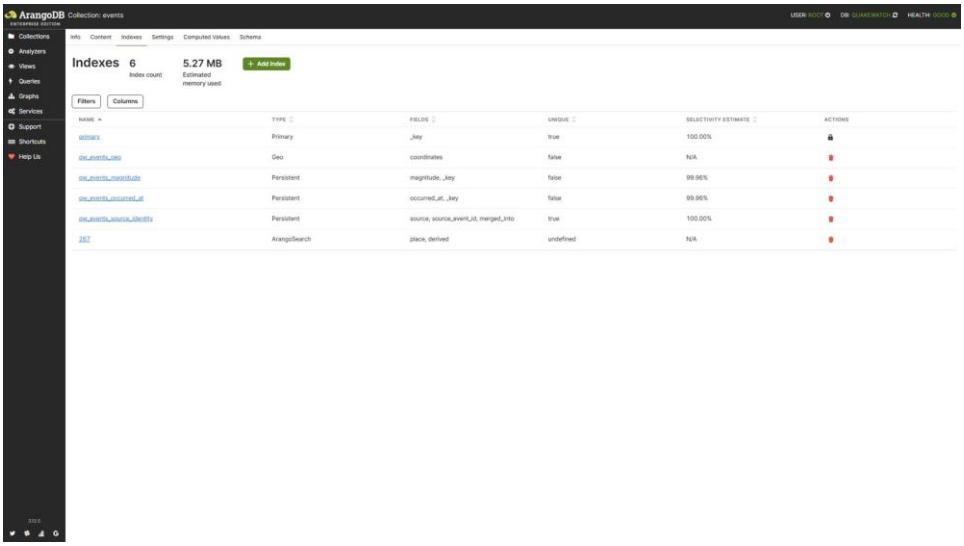


Figure 7: Indexes tab of the events collection: primary, geo and persistent indexes and the search link.

2.6 Views

The ArangoSearch view events_search links the events collection and indexes the place and derived region fields with text analyzers; its primary sort is occurred_at in descending order, so the newest matches are read first. Query 12 uses it for tokenised location search (Figure 8).

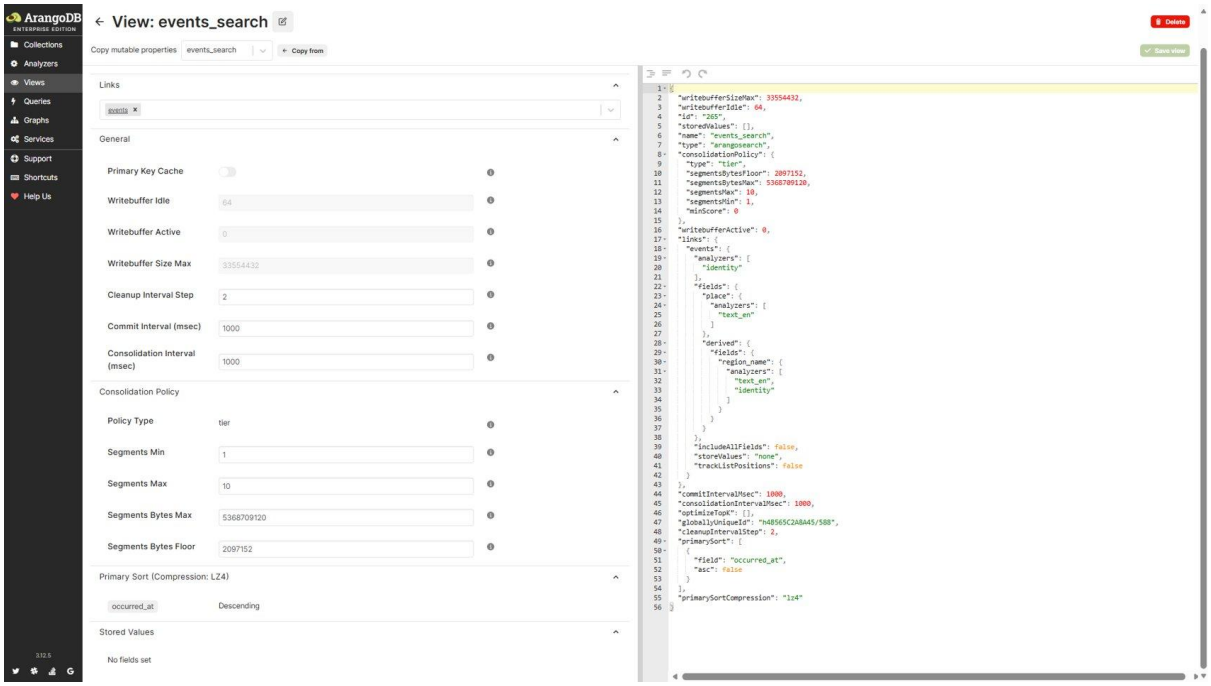


Figure 8: ArangoSearch view events_search: links, analyzers and primary sort.

2.7 Triggers and procedures: NoSQL equivalents

ArangoDB has no SQL-style triggers and no stored procedures, and the AQL user-function registry of this database is empty (verified through the HTTP API). The behaviour they provide in a relational system is obtained from the mechanisms below, each of which was exercised against the live database.

Table 6: Relational constructs and their ArangoDB counterparts

Relational concept	Mechanism in QuakeWatch	Where
BEFORE INSERT trigger / CHECK	Strict JSON-schema validation; rejected writes fail with error 1620	ArangoDB server
UNIQUE constraint	Unique persistent indexes; violations fail with error 1210	ArangoDB server
Scheduled clean-up	TTL indexes on sessions, cache, rate limits, raw payloads, leases, jobs	ArangoDB server
AFTER UPDATE trigger	Ingestion pipeline calls registered observers (for example the alert-rule observer) after an event changes	Application
Stored procedure	53 named parameterised AQL queries executed with bind variables only	Application
BEGIN / COMMIT / ROLLBACK	Stream transactions with automatic retry on write-write conflicts (error 1200)	Application

Table 7: Integrity mechanisms exercised on the live database (counts of events and audit_log unchanged afterwards)

Demonstration	Action	Observed result
Schema validation	Insert an event with an invalid source and magnitude 99	Rejected, error 1620
Unique index	Insert a copy of an existing event under a new key	Rejected, error 1210 (qw_events_source_identity)
Transaction rollback	One transaction: a valid audit write followed by an invalid event write	Aborted, error 1620; the valid write was rolled back
Transaction commit	One transaction with a valid audit write	Committed and visible (removed afterwards)

2.8 Database export (dump)

The native tool arangodump writes the structure and data of every collection (structure and gzip data files, a manifest and the view definition). A dump of an isolated test database was generated on 11 October 2026 and verified readable (31 collections, 32.8 KB compressed); it is stored under verification_evidence/review2_2026-10-11. The live database can be exported with the command below. No restore test was performed.

```
docker compose exec arangodb arangodump --server.database quakewatch --output-
directory /tmp/qw_dump
```

3. Complex AQL Queries with Output

Fifteen queries from the application's named-query catalogue were executed against the live database in the ArangoDB query editor. Together they cover aggregation, subqueries, joins by document lookup, array-index lookups, graph traversals in both directions, geospatial search, ArangoSearch and time-window analytics. For each query the screenshots show the AQL with its bind variables and the returned result; the observation states what the result demonstrates. Because the feed is live, counts can differ slightly between screenshots.

3.1 Query 1: Event count and magnitude statistics per region

Purpose: Event count and magnitude statistics per region.

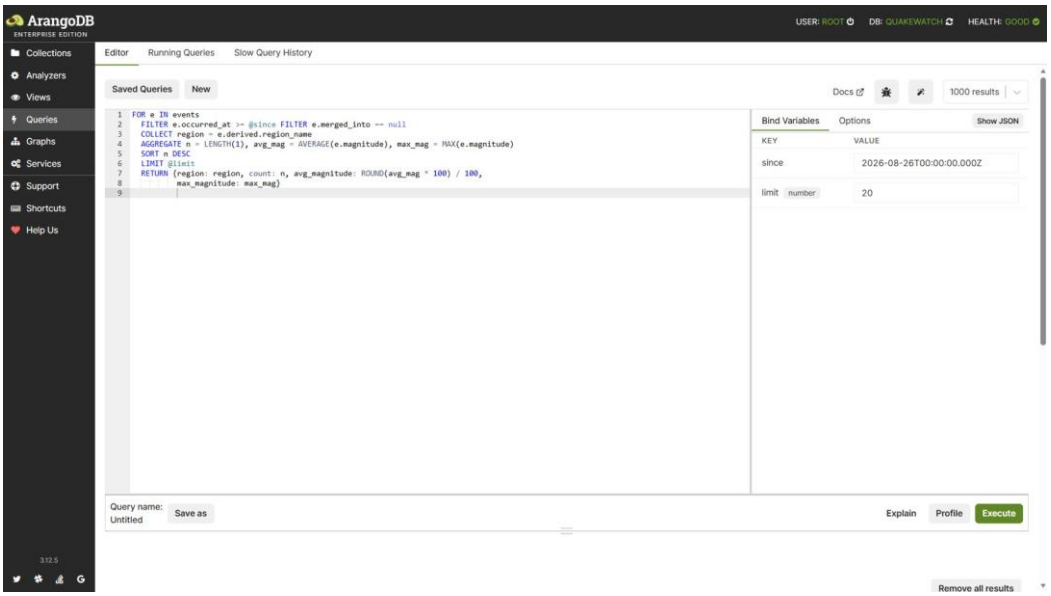


Figure 9: Query 1: AQL editor with bind variables.

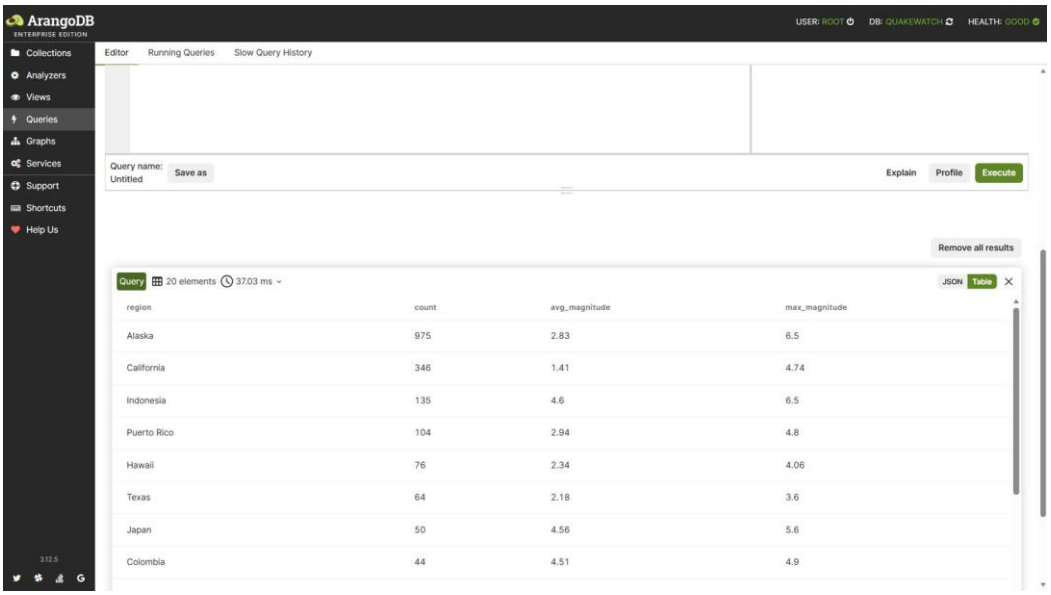


Figure 10: Query 1: returned result.

Observation: Aggregation ranked 20 regions in 37 ms; Alaska led with 975 events, followed by California (346) and Indonesia (135). Index chosen by the optimizer: qw_events_occurred_at.

3.2 Query 2: Dashboard summary with subqueries

Purpose: Dashboard summary with several subqueries.

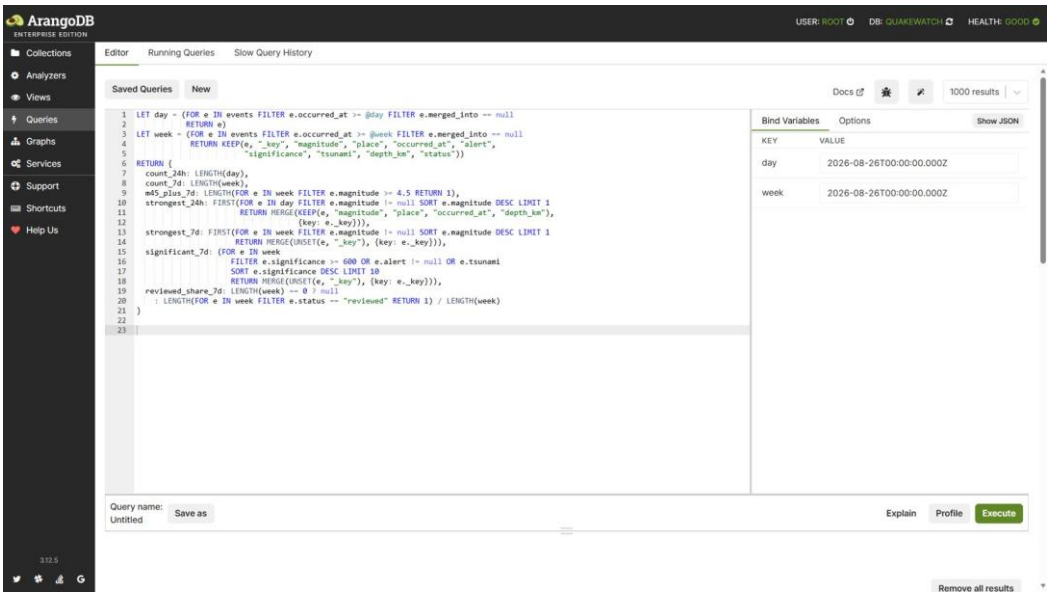


Figure 11: Query 2: AQL editor with bind variables.

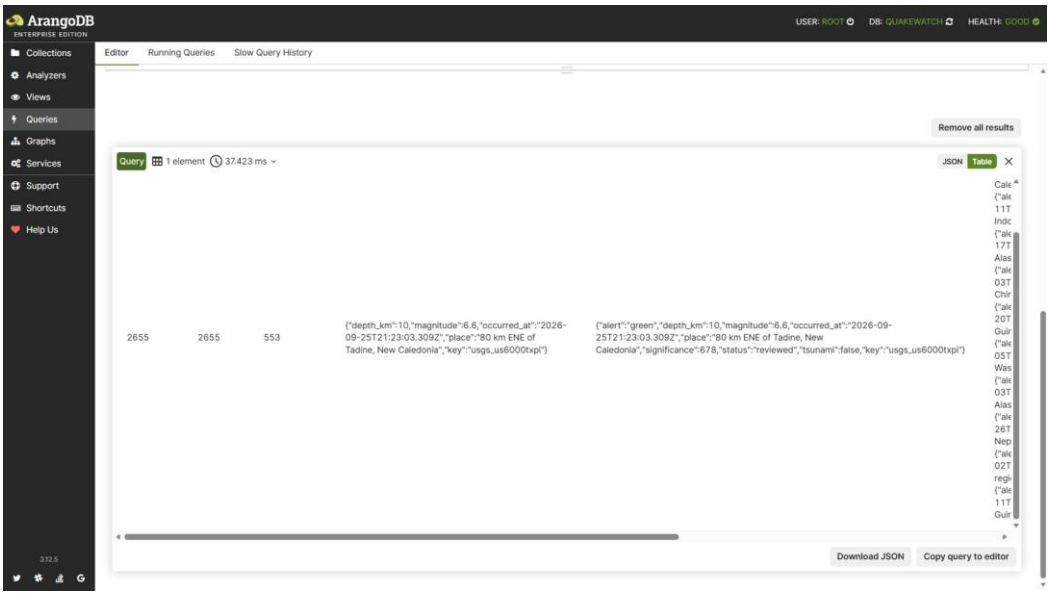


Figure 12: Query 2: returned result.

Observation: One document returned the 24-hour and 7-day counts (2,655), 553 events of M4.5 or more and the strongest event, M6.6 near Tadine, New Caledonia. Index chosen by the optimizer: qw_events_occurred_at.

3.3 Query 3: Per-network statistics joined to network names

Purpose: Per-network statistics joined to network names.

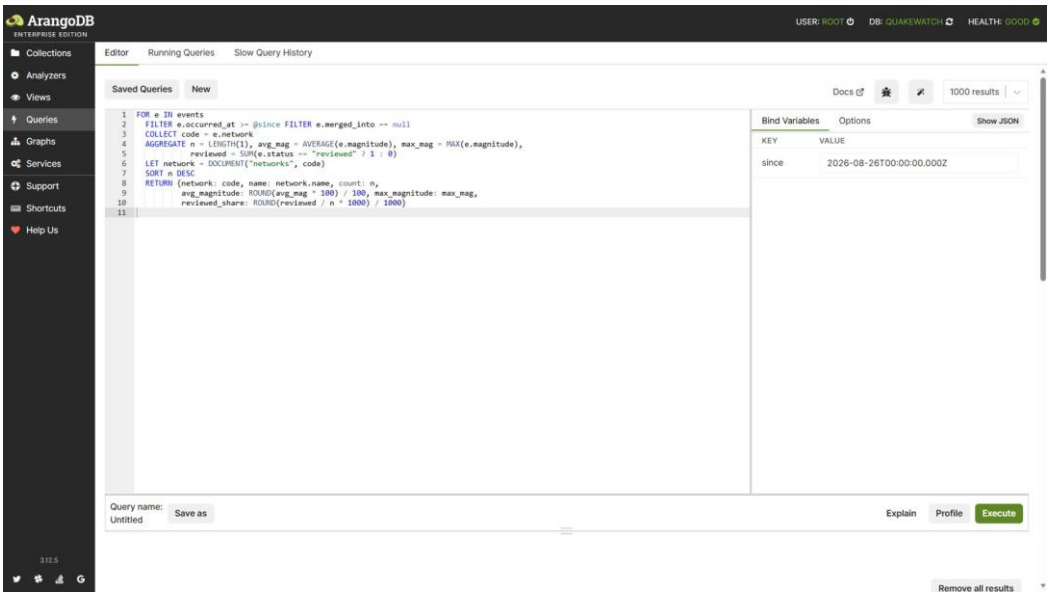


Figure 13: Query 3: AQL editor with bind variables.

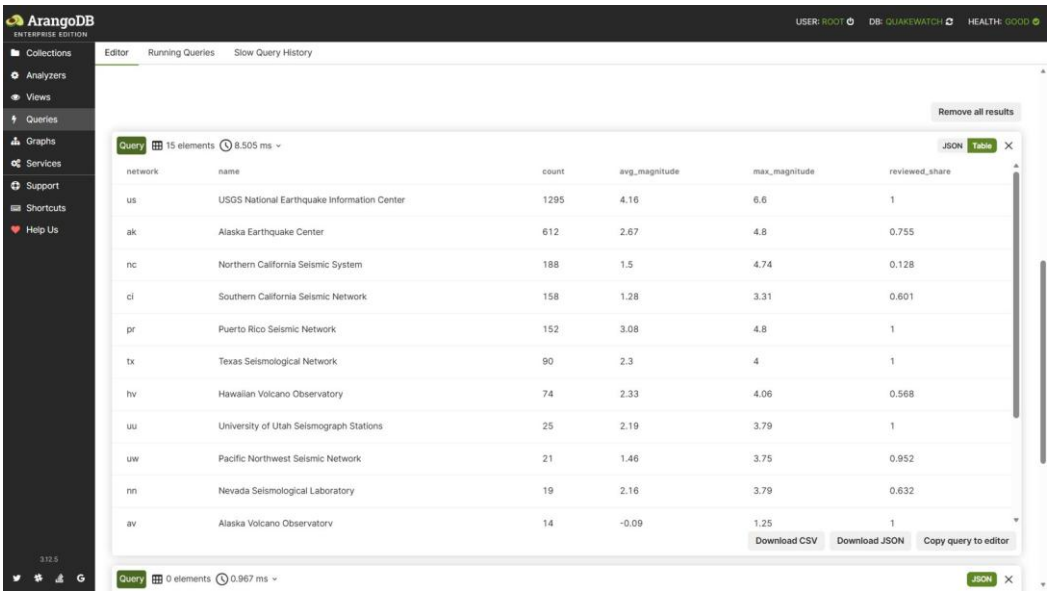


Figure 14: Query 3: returned result.

Observation: Fifteen networks returned in 8.5 ms; the USGS national network (us) reported 1,295 events and Alaska Earthquake Center 612, with names joined by document lookup. Index chosen by the optimizer: qw_events_occurred_at.

3.4 Query 4: Revisions with large magnitude changes (array index)

Purpose: Revisions whose magnitude changed by at least 0.3.

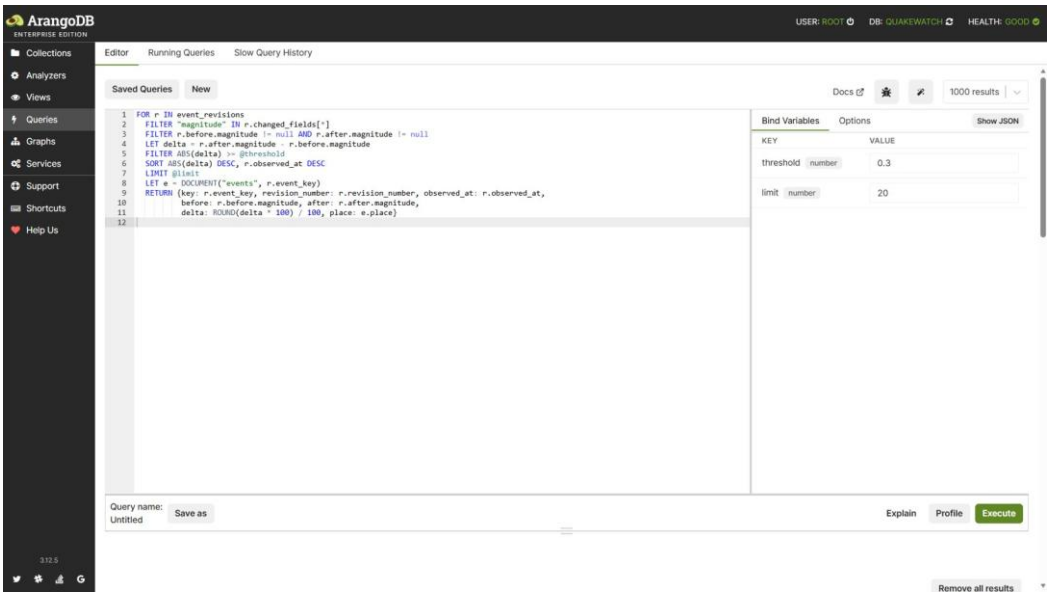


Figure 15: Query 4: AQL editor with bind variables.

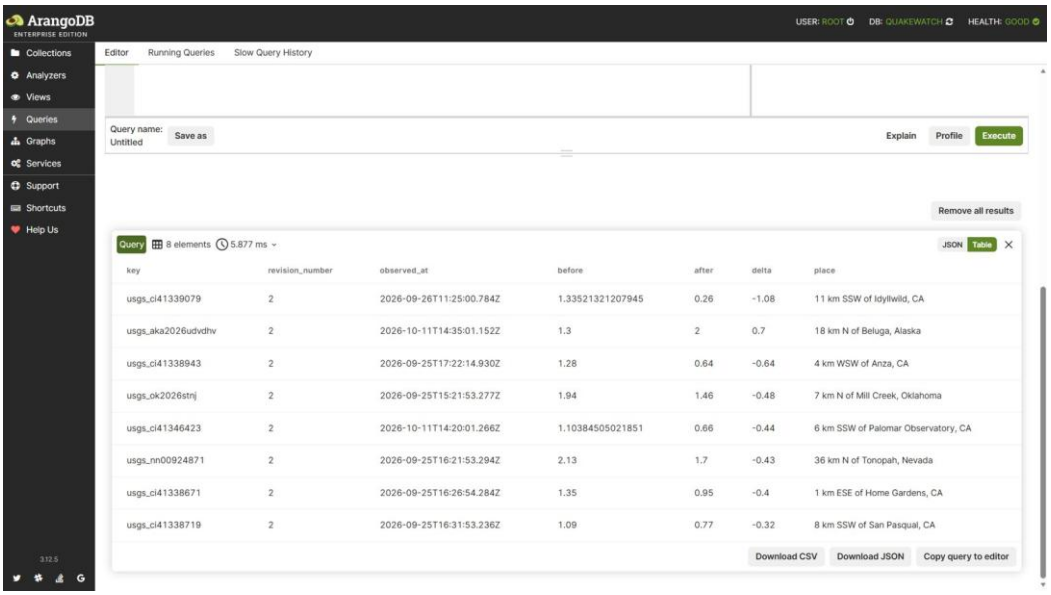


Figure 16: Query 4: returned result.

Observation: Eight revisions found through the array index; the largest change was -1.08 for usgs_ci41339079, showing that revision history is queryable. Index chosen by the optimizer: qw_revisions_changed_fields.

3.5 Query 5: Ingestion latency percentiles

Purpose: First-seen latency percentiles of ingested events.

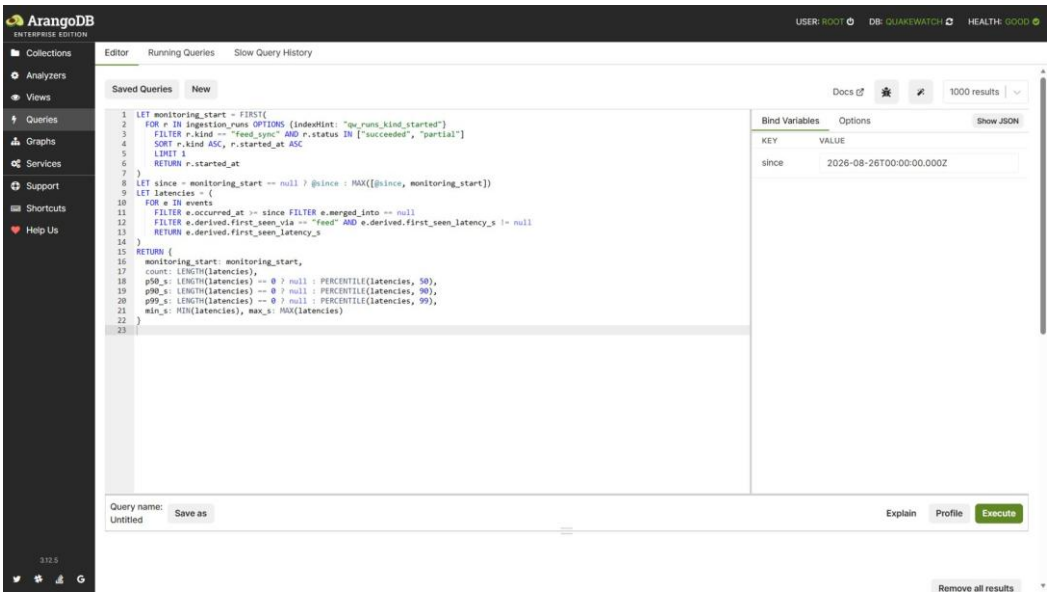


Figure 17: Query 5: AQL editor with bind variables.

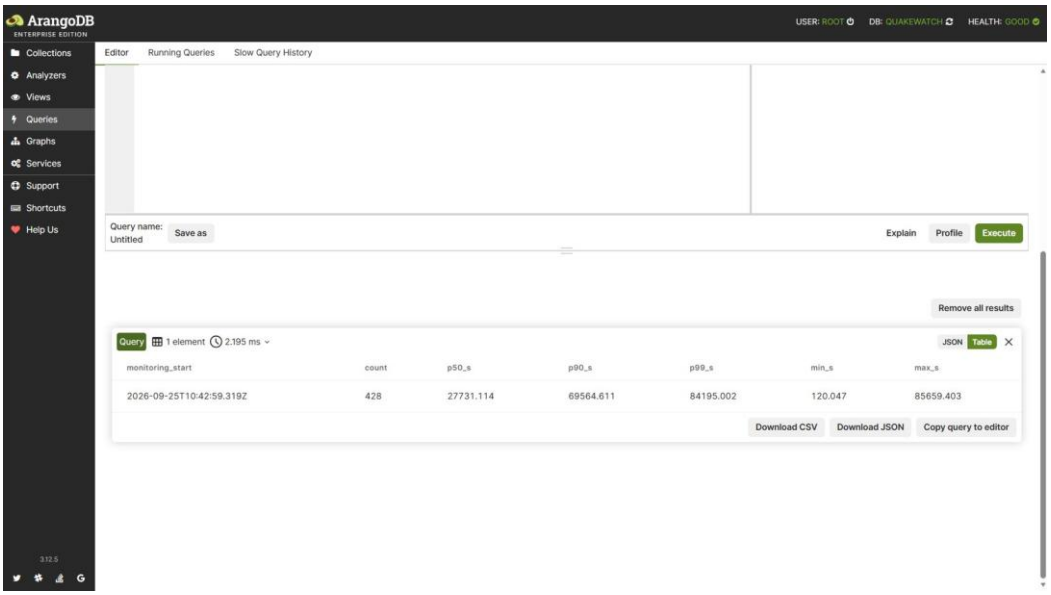


Figure 18: Query 5: returned result.

Observation: Percentile functions over 428 events gave p50 27,731 s and p99 84,195 s in 2.2 ms. Index chosen by the optimizer: qw_events_occurred_at, qw_runs_kind_started.

3.6 Query 6: Significant events per ISO week

Purpose: Significant events grouped by ISO week.

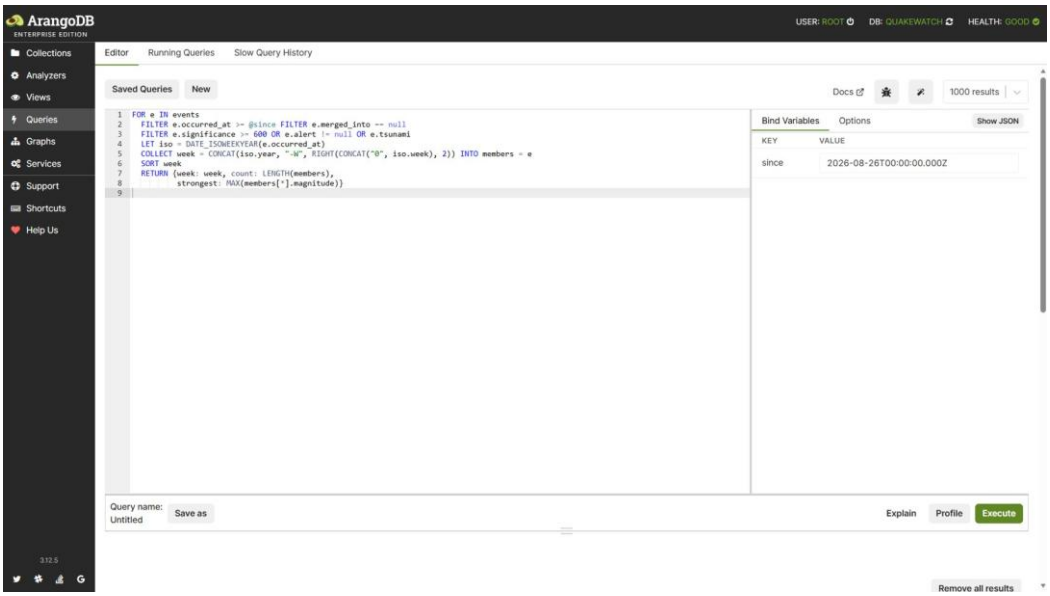


Figure 19: Query 6: AQL editor with bind variables.

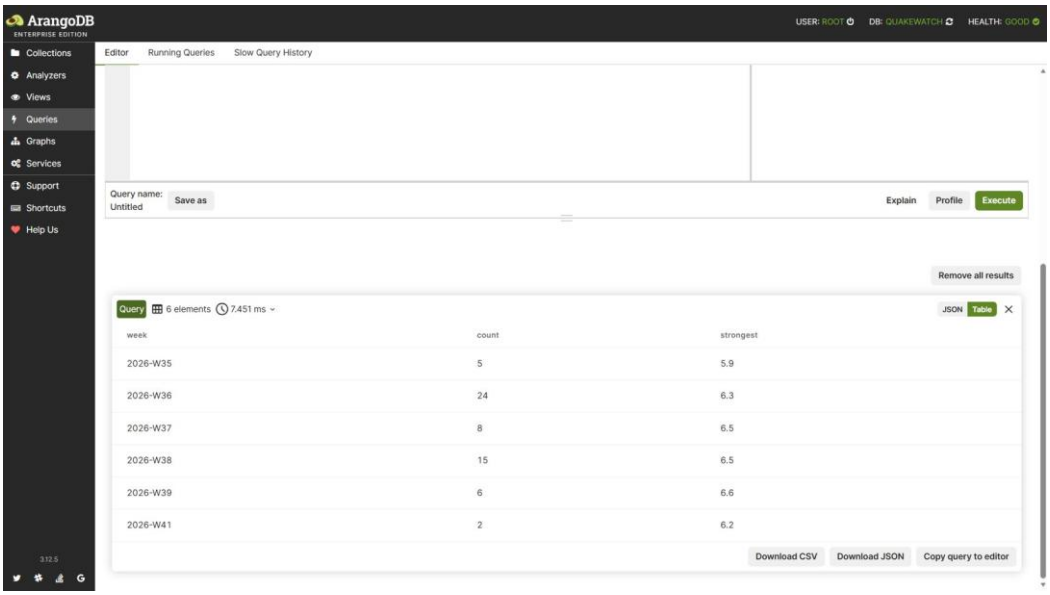


Figure 20: Query 6: returned result.

Observation: Six weeks (2026-W35 to W41) returned in 7.5 ms; the strongest weekly magnitude was 6.6 in week 39. Index chosen by the optimizer: qw_events_occurred_at.

3.7 Query 7: A user's alert rules with match counts (edge traversal)

Purpose: Alert rules of one user with match counts (edge traversal).

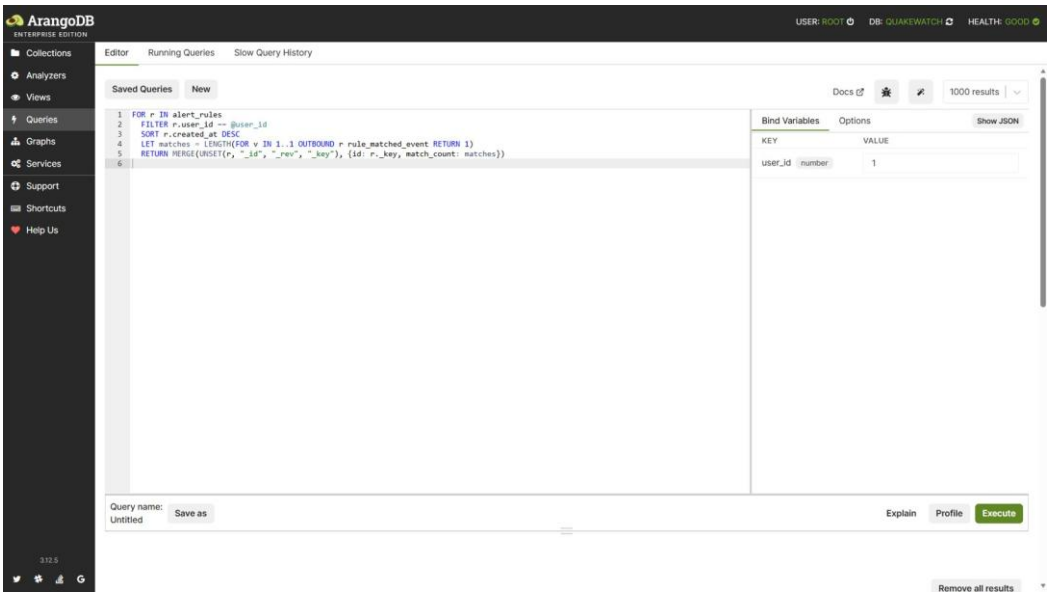


Figure 21: Query 7: AQL editor with bind variables.

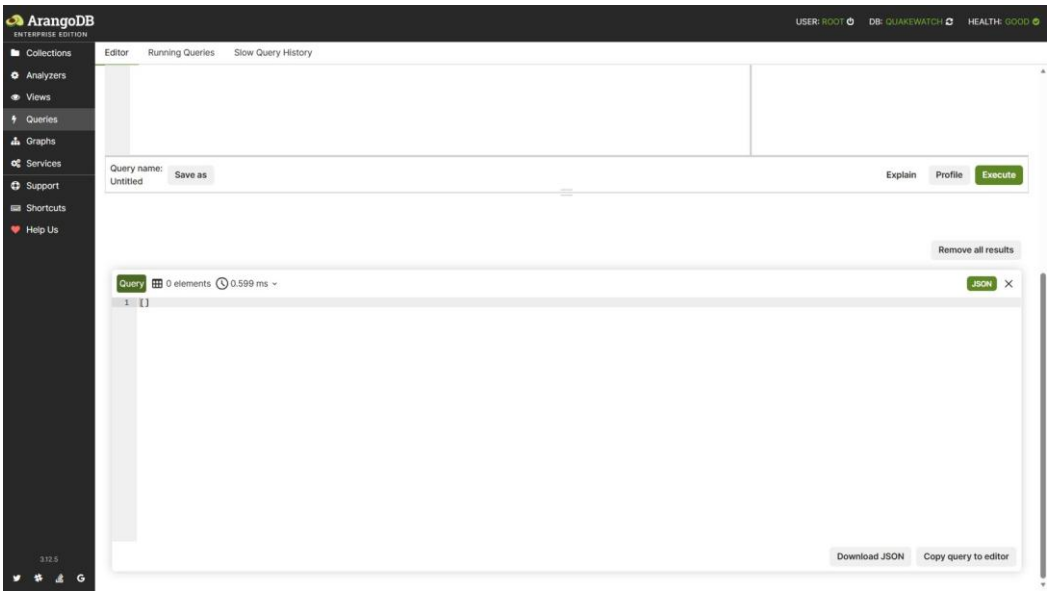


Figure 22: Query 7: returned result.

Observation: The screenshot used user 1, who owned no rules at that moment, so the result is empty (0 rows, 0.6 ms); with a rule present the query returned the rule and its match count. Index chosen by the optimizer: edge, qw_rules_user.

3.8 Query 8: Matched events through a two-hop graph traversal

Purpose: Matched events through a two-hop traversal (user, rule, event).

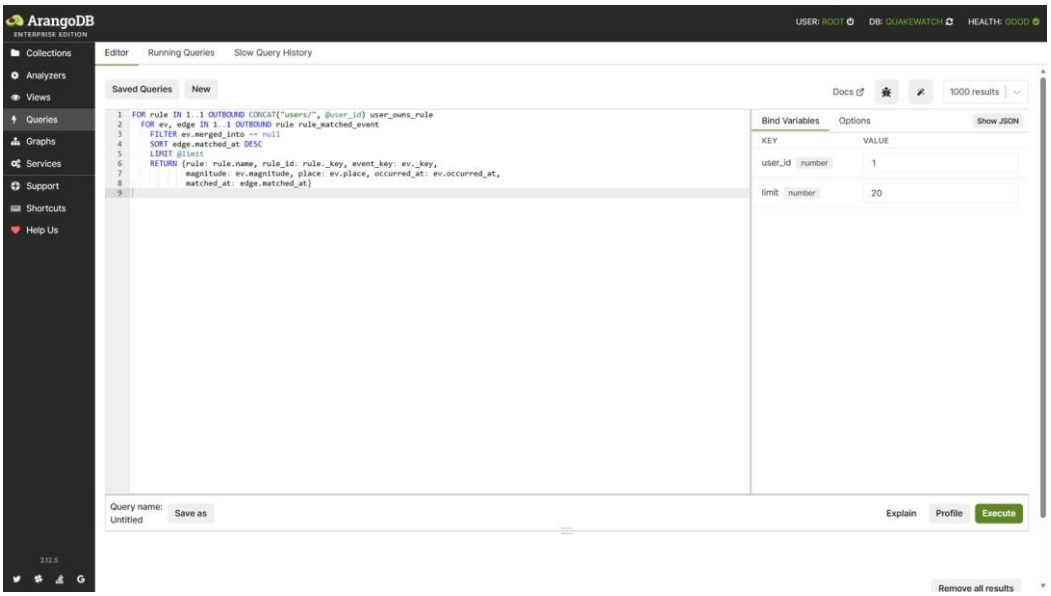


Figure 23: Query 8: AQL editor with bind variables.

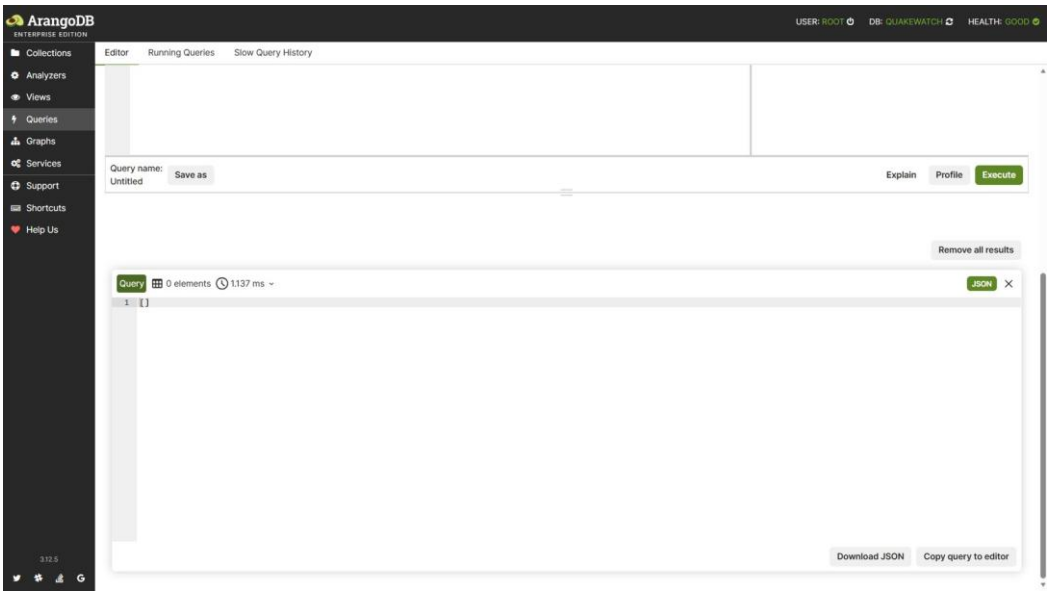


Figure 24: Query 8: returned result.

Observation: Empty result for the same user (0 rows, 1.1 ms); with a matching rule the traversal returned the matched event with its rule name. Index chosen by the optimizer: edge.

3.9 Query 9: Event relationships (OUTBOUND and INBOUND traversal)

Purpose: Relationships of one event through OUTBOUND and INBOUND traversals.

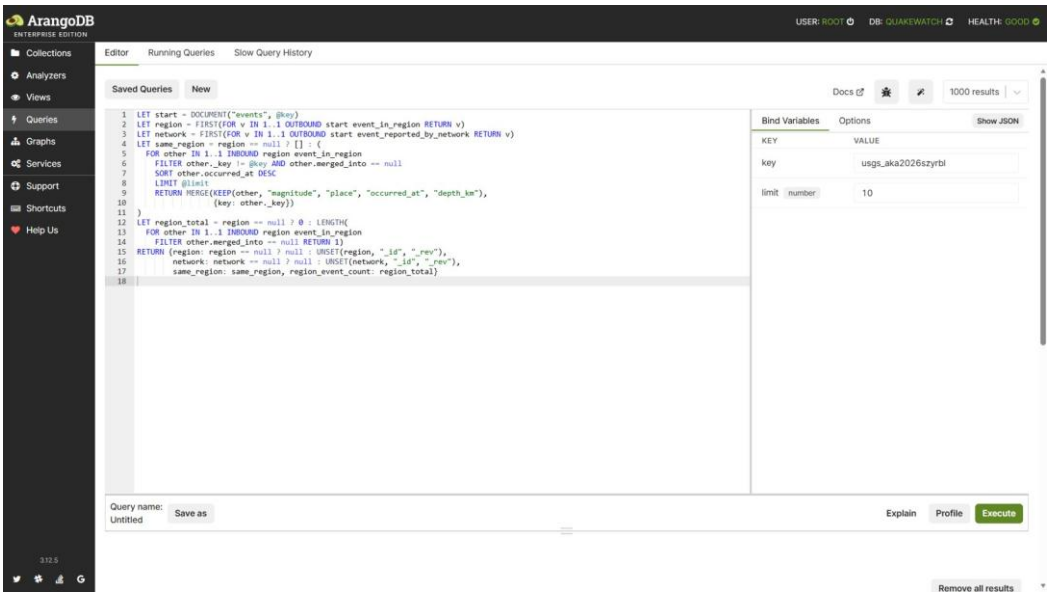


Figure 25: Query 9: AQL editor with bind variables.

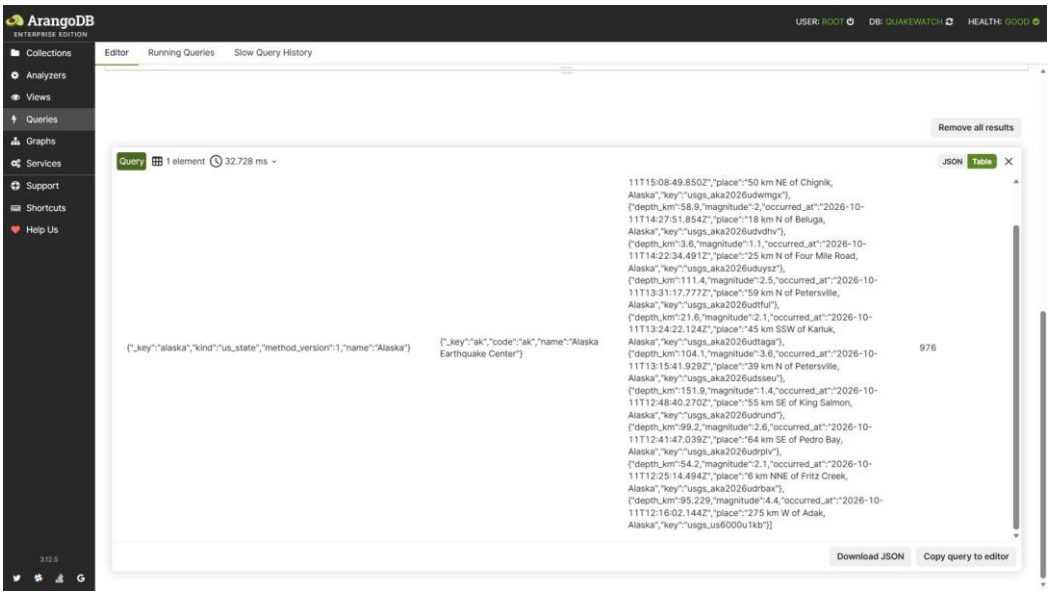


Figure 26: Query 9: returned result.

Observation: For event usgs_aka2026szyrbl the traversal returned its region (Alaska), its network (Alaska Earthquake Center) and 976 events in the same region in 32.7 ms. Index chosen by the optimizer: edge.

3.10 Query 10: Map viewport search (GEO_CONTAINS)

Purpose: Map viewport search with GEO_CONTAINS.

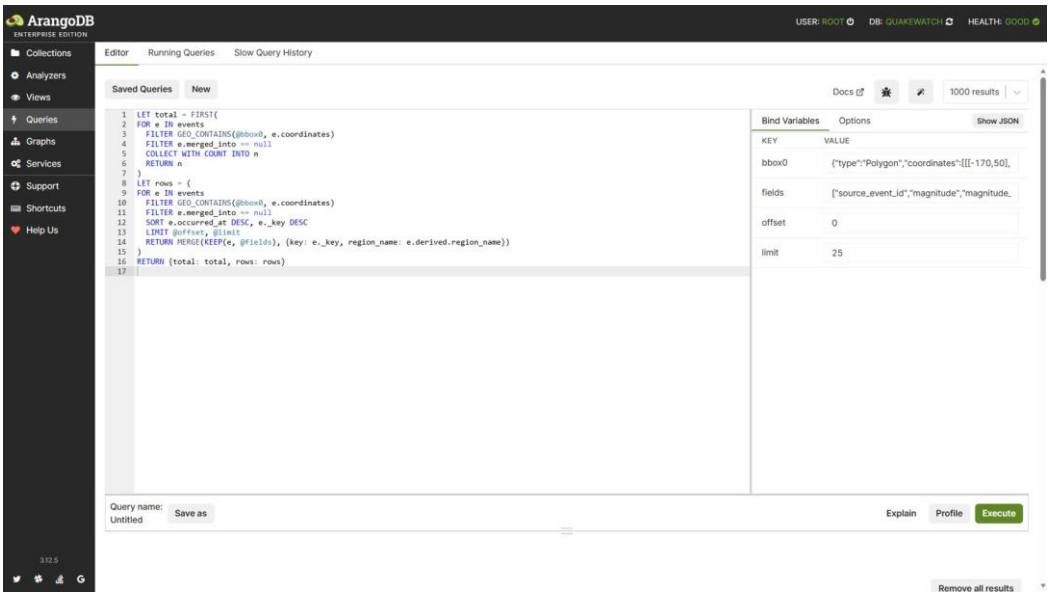


Figure 27: Query 10: AQL editor with bind variables.

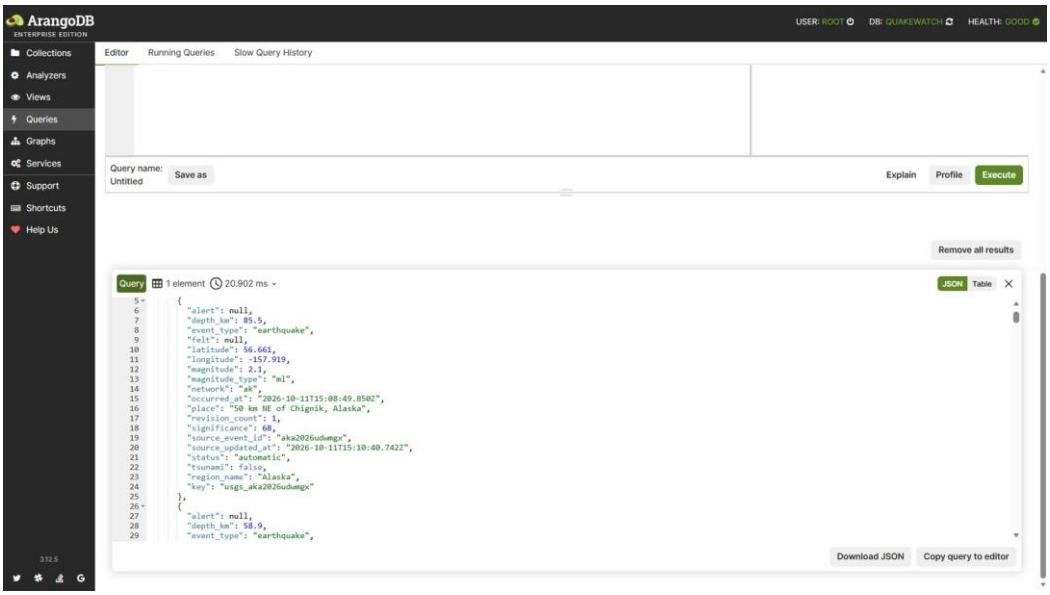


Figure 28: Query 10: returned result.

Observation: The geo index returned the event total inside the polygon and the first page of events in 20.9 ms. Index chosen by the optimizer: qw_events_geo.

3.11 Query 11: Radius search (GEO_DISTANCE)

Purpose: Radius search with GEO_DISTANCE.

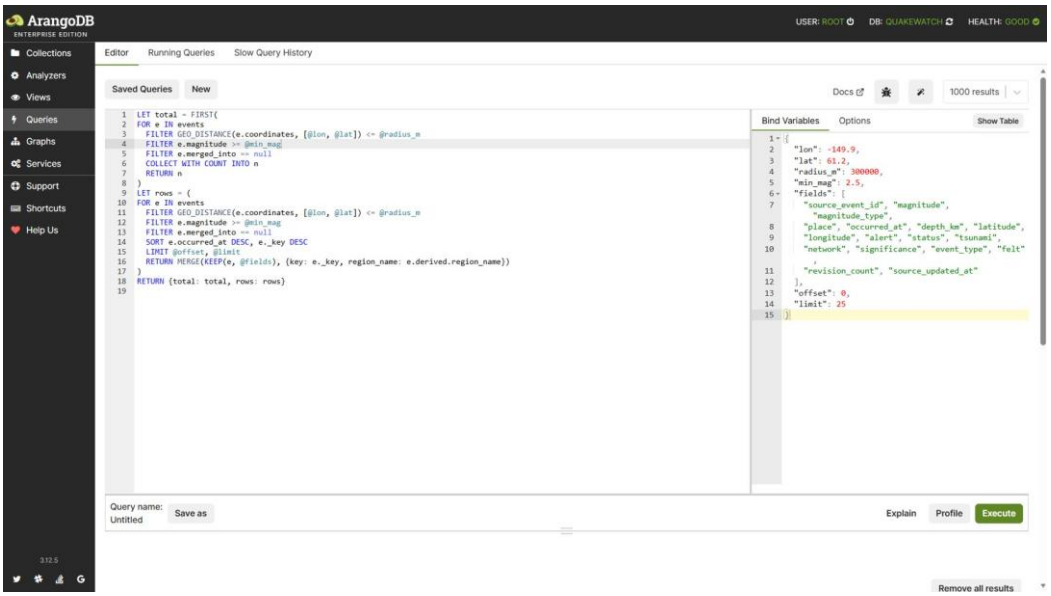


Figure 29: Query 11: AQL editor with bind variables.

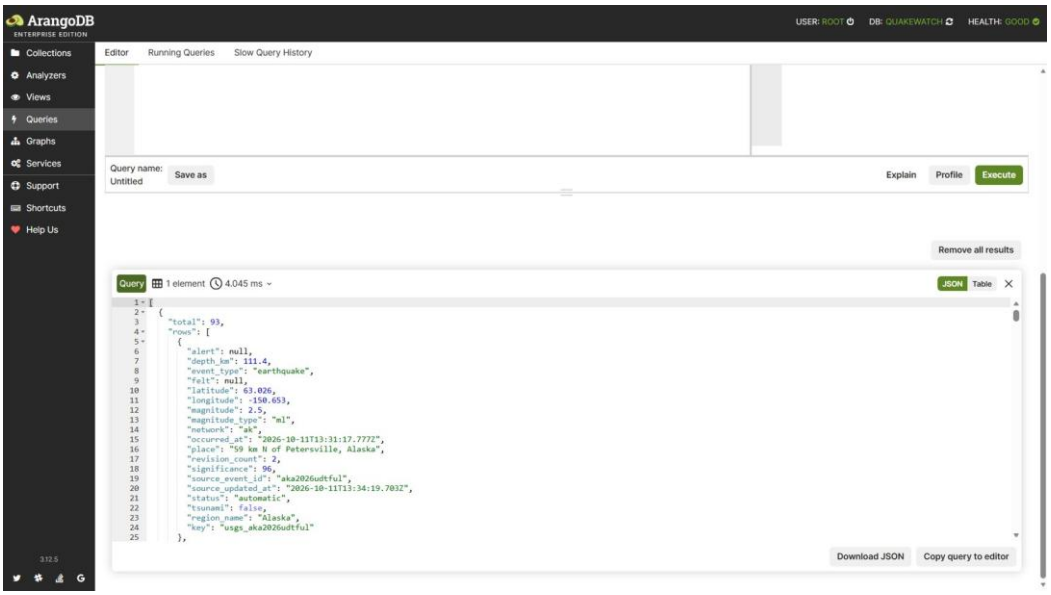


Figure 30: Query 11: returned result.

Observation: 93 events of M2.5 or more within 300 km of Anchorage were found in 4.0 ms through the geo index. Index chosen by the optimizer: qw_events_geo.

3.12 Query 12: Location text search (ArangoSearch view)

Purpose: Location text search through the ArangoSearch view.

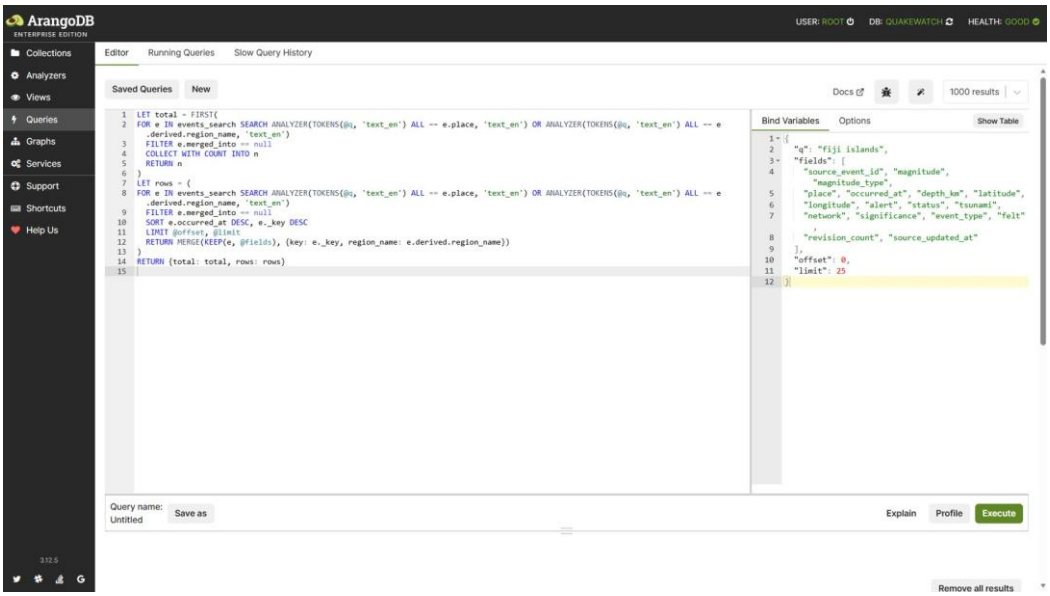


Figure 31: Query 12: AQL editor with bind variables.

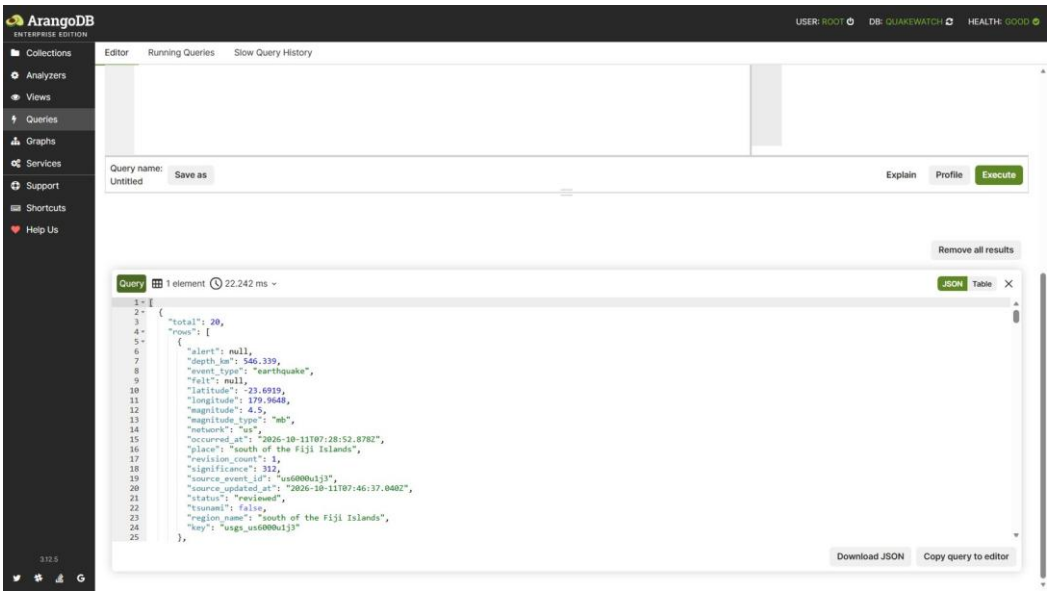


Figure 32: Query 12: returned result.

Observation: The text query 'fiji islands' matched 20 events in 22.2 ms using the text_en analyzer. Index chosen by the optimizer: ArangoSearch view.

3.13 Query 13: Ingestion run statistics by kind

Purpose: Ingestion run statistics by kind.

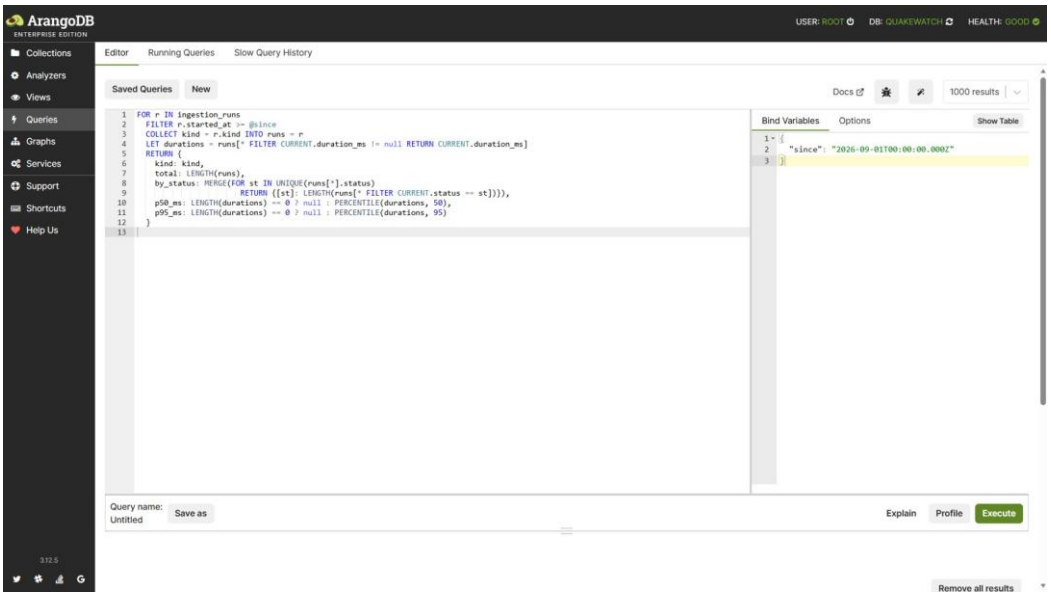


Figure 33: Query 13: AQL editor with bind variables.

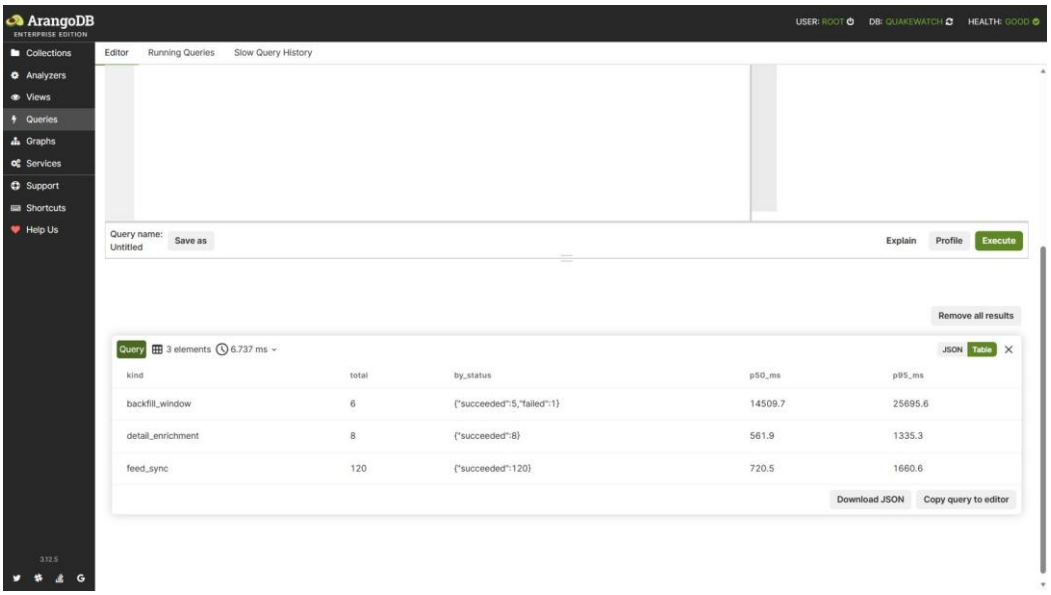


Figure 34: Query 13: returned result.

Observation: Three run kinds were summarised with status mix and p50/p95 durations; feed synchronisation had a median of about 721 ms. Index chosen by the optimizer: qw_runs_kind_started.

3.14 Query 14: Depth histogram (bucketed aggregation)

Purpose: Depth histogram in 10 km bins.

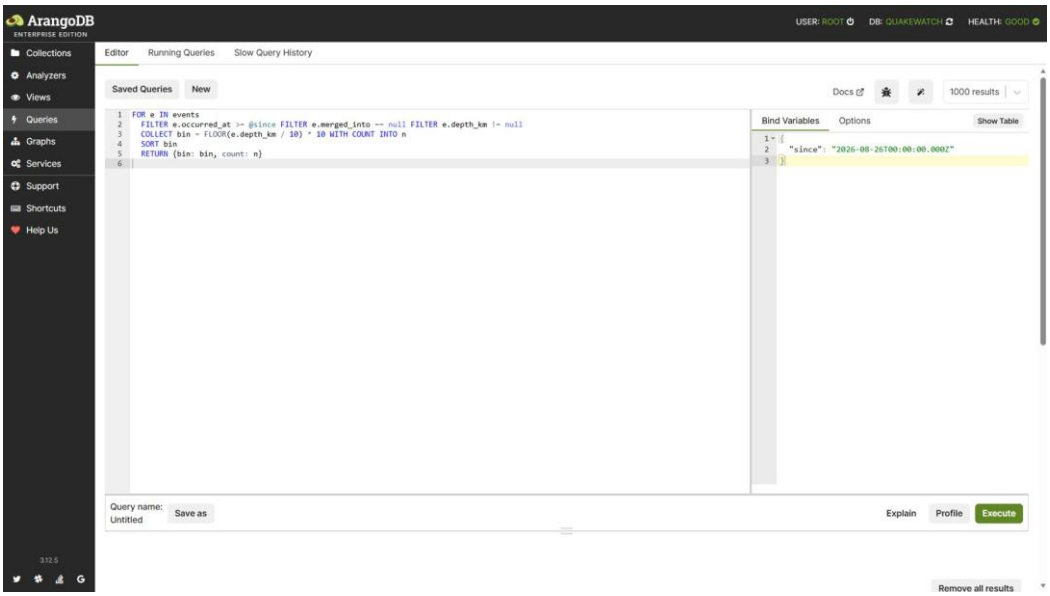


Figure 35: Query 14: AQL editor with bind variables.

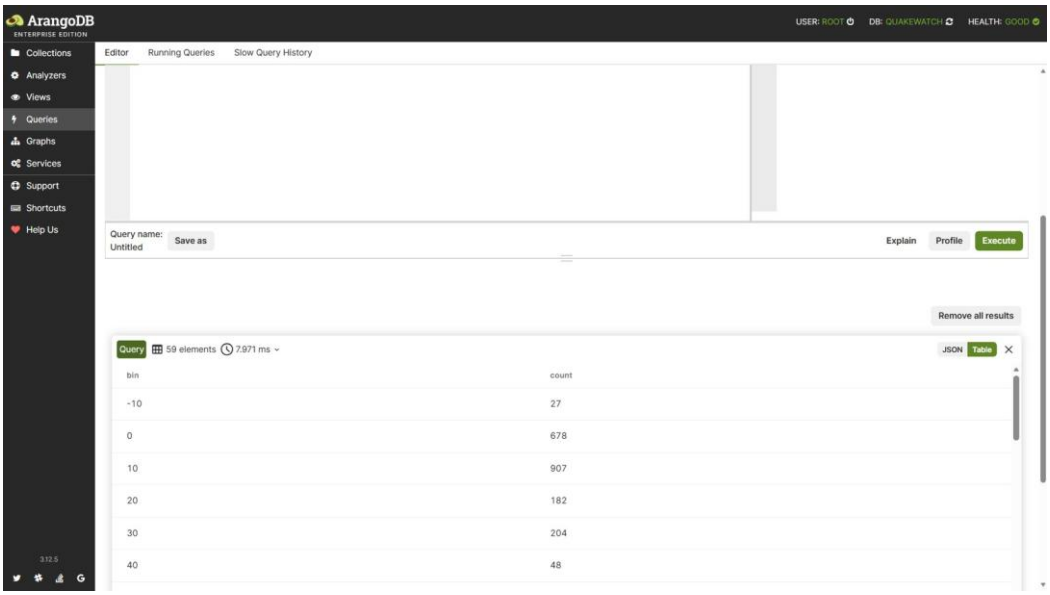


Figure 36: Query 14: returned result.

Observation: 59 bins in 8.0 ms; most events fall in the 0 to 20 km range (678 and 907 events). Index chosen by the optimizer: qw_events_occurred_at.

3.15 Query 15: Nearby events in space and time

Purpose: Nearby events in space and time.

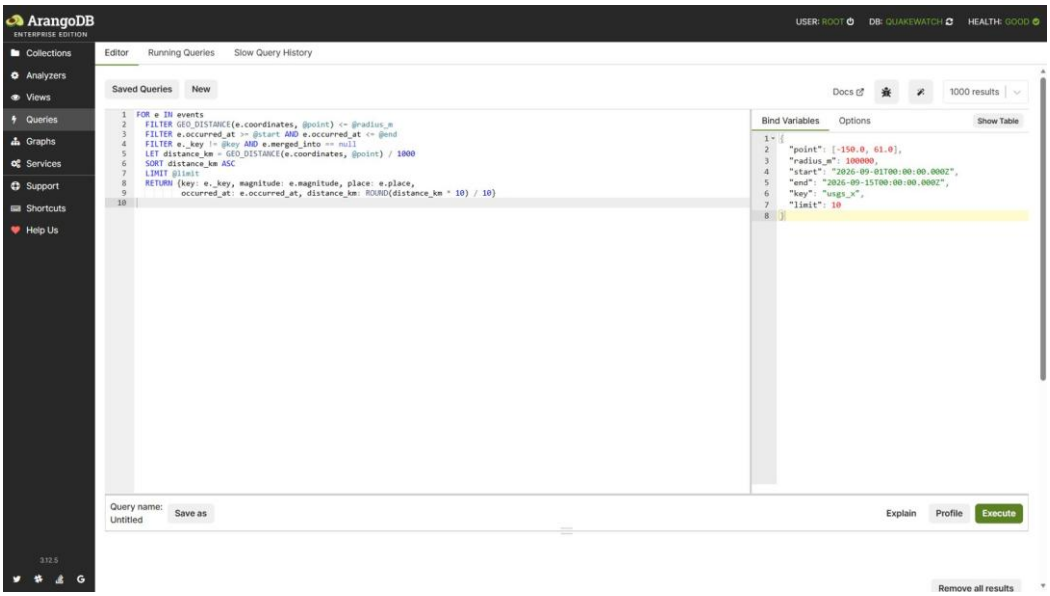


Figure 37: Query 15: AQL editor with bind variables.

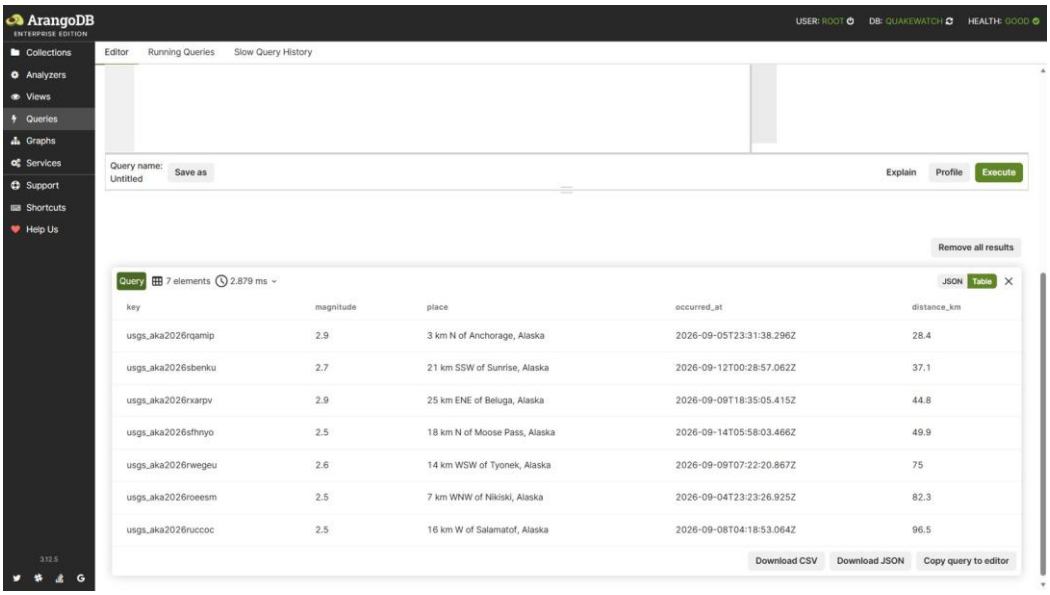


Figure 38: Query 15: returned result.

Observation: Seven events within 100 km of the reference point and the time window were returned in 2.9 ms, nearest at 28.4 km. Index chosen by the optimizer: qw_events_geo.

3.16 Summary of the query execution

Table 8 lists the row counts, times and indexes of a scripted re-run of the same fifteen queries on the live database; the optimizer's index choice was read from EXPLAIN.

Table 8: Fifteen AQL queries re-executed against the live database

#	Query	Rows	Time (ms)	Index used
1	analytics_avg_magnitude_by_region	20	6.98	qw_events_occurred_at
2	analytics_dashboard_summary	1	32.24	qw_events_occurred_at
3	analytics_network_stats	15	6.83	qw_events_occurred_at
4	analytics_magnitude_changes	10	3.15	qw_revisions_changed_fields
5	analytics_ingestion_latency	1	3.29	qw_events_occurred_at, qw_runs_kind_started
6	analytics_significant_by_week	6	9.78	qw_events_occurred_at
7	alerts_user_rules	1	2.18	edge, qw_rules_user
8	alerts_matched_events_graph	1	1.22	edge
9	event_graph_relationships	1	31.85	edge
10	events_in_viewport	1	9.77	qw_events_geo
11	events_within_radius	1	3.52	qw_events_geo
12	events_location_text	1	4.11	ArangoSearch view
13	ops_run_stats	3	2.04	qw_runs_kind_started
14	analytics_depth_histogram	59	8.96	qw_events_occurred_at
15	event_nearby_space_time	7	1.56	qw_events_geo

4. Backend and Database Connectivity: CRUD Operations

4.1 Connectivity

The Django REST API served on localhost:8000 reaches ArangoDB through the python-arango driver. Connectivity was verified in three ways: the health endpoint reported the database and cache as up with schema version 6 applied (6 of 6); authenticated requests created, changed and removed documents; and after each write or delete the stored document was read back directly from ArangoDB to confirm the change. Requests were authenticated through the application's session mechanism.

Source code (GitHub repository):

https://github.com/abharathreddy751/quake_watch_v2.git

4.2 CRUD operations demonstrated

The seven operations of Table 9 were executed in order in Postman against the running application, using one resource (a saved search) so that create, read, update and delete can be seen on the same record. The status codes below were recorded on 11 October 2026.

Table 9: CRUD operations executed through the API (saved-search resource)

S.No.	CRUD Operation	HTTP Method	HTTP Status Code	Result
1	Create saved search	POST	201	Success
2	List saved searches	GET	200	Success
3	Read saved search by id	GET	200	Success
4	Update (rename) saved search	PATCH	200	Success
5	Run saved search on earthquake data	GET	200	Success
6	Delete saved search	DELETE	204	Success
7	Verify deletion	GET	404	Success (404 expected: deletion confirmed)

A separate automated run of fifteen requests additionally verified creating, updating and deleting an alert rule and a research snapshot; those results are listed in Section 5.

CRUD operations performance video:

https://drive.google.com/file/d/13xG6iQIpoML8b9e8bDhqCjgQHI9wToLa/view?usp=drive_link

5. Test Cases and Results

Each test was run against the live application and database. Where a test failed first and was repaired or corrected and retested, both outcomes are recorded.

Table 10: Executed test cases (38)

Test ID	Test Case	Expected Result	Actual Result	Pass/Fail
TC-01	API: Open login page (obtain CSRF cookie)	HTTP 200	HTTP 200	Pass
TC-02	API: Log in (create session)	HTTP 302	HTTP 302	Pass
TC-03	API: System health (backend + DB connectivity)	HTTP 200	HTTP 200	Pass
TC-04	API: List and filter earthquake events	HTTP 200	HTTP 200	Pass
TC-05	API: Create saved search	HTTP 201	HTTP 201	Pass
TC-06	API: Read saved search (persistence check)	HTTP 200	HTTP 200	Pass
TC-07	API: Rename saved search	HTTP 200	HTTP 200	Pass
TC-08	API: Create alert rule	HTTP 201	HTTP 201	Pass
TC-09	API: Update alert rule	HTTP 200	HTTP 200	Pass
TC-10	API: Preview alert rule against stored events	HTTP 200	HTTP 200	Pass
TC-11	API: Create research snapshot	HTTP 201	HTTP 201	Pass
TC-12	API: Delete research snapshot	HTTP 204	HTTP 204	Pass
TC-13	API: Delete alert rule	HTTP 204	HTTP 204	Pass
TC-14	API: Delete saved search	HTTP 204	HTTP 204	Pass
TC-15	API: Verify deletion (expect 404) (negative)	HTTP 404	HTTP 404	Pass
TC-16	Postman: seven-operation saved-search CRUD flow	Statuses 201, 200, 200, 200, 200, 204, 404	Statuses 201, 200, 200, 200, 200, 204, 404	Pass
TC-17	AQL: Query 1 (analytics_avg_magnitude_by_region)	Executes, returns rows, uses expected index	20 row(s) in 6.98 ms	Pass
TC-18	AQL: Query 2 (analytics_dashboard_summary)	Executes, returns rows, uses expected index	1 row(s) in 32.24 ms	Pass
TC-19	AQL: Query 3 (analytics_network_stats)	Executes, returns rows, uses expected index	15 row(s) in 6.83 ms	Pass

Test ID	Test Case	Expected Result	Actual Result	Pass/Fail
TC-20	AQL: Query 4 (analytics_magnitude_changes)	Executes, returns rows, uses expected index	10 row(s) in 3.15 ms	Pass
TC-21	AQL: Query 5 (analytics_ingestion_latency)	Executes, returns rows, uses expected index	1 row(s) in 3.29 ms	Pass
TC-22	AQL: Query 6 (analytics_significant_by_week)	Executes, returns rows, uses expected index	6 row(s) in 9.78 ms	Pass
TC-23	AQL: Query 7 (alerts_user_rules)	Executes, returns rows, uses expected index	1 row(s) in 2.18 ms	Pass
TC-24	AQL: Query 8 (alerts_matched_events_graph)	Executes, returns rows, uses expected index	1 row(s) in 1.22 ms	Pass
TC-25	AQL: Query 9 (event_graph_relationships)	Executes, returns rows, uses expected index	1 row(s) in 31.85 ms	Pass
TC-26	AQL: Query 10 (events_in_viewport)	Executes, returns rows, uses expected index	1 row(s) in 9.77 ms	Pass
TC-27	AQL: Query 11 (events_within_radius)	Executes, returns rows, uses expected index	1 row(s) in 3.52 ms	Pass
TC-28	AQL: Query 12 (events_location_text)	Executes, returns rows, uses expected index	1 row(s) in 4.11 ms	Pass
TC-29	AQL: Query 13 (ops_run_stats)	Executes, returns rows, uses expected index	3 row(s) in 2.04 ms	Pass
TC-30	AQL: Query 14 (analytics_depth_histogram)	Executes, returns rows, uses expected index	59 row(s) in 8.96 ms	Pass
TC-31	AQL: Query 15 (event_nearby_space_time)	Executes, returns rows, uses expected index	7 row(s) in 1.56 ms	Pass
TC-32	Integrity: insert schema-invalid event (negative)	Rejected by validator	Error 1620	Pass
TC-33	Integrity: insert duplicate event identity (negative)	Unique constraint violation	Error 1210	Pass
TC-34	Transaction: valid and invalid write together (negative)	Whole transaction rolled back	Aborted; no partial write	Pass
TC-35	Transaction: valid write in a stream transaction	Committed and visible	Committed and visible	Pass
TC-36	Sign-up with a password equal to the username (negative)	Validation error, HTTP 400	Initial run: HTTP 500 (server error). After repair of the form validator and retest: HTTP 400 with message	Fail, then Pass after repair
TC-37	Postman/Newman run of the 15-request collection	0 failed assertions	Initial run failed (login URL lost its trailing slash; stale CSRF value). After correcting the collection: 15 requests, 35 assertions, 0 failed	Fail, then Pass after correction
TC-38	Automated regression suite (pytest, 936 selected tests)	All tests pass	935 passed, 1 failed (test_production_requires_long_secret: container inherited QW_DEBUG=true). Retest with the variable unset: passed	Fail (environment), then Pass on retest

The regression suite covers unit, contract, integration (real ArangoDB test database), resilience and architecture-boundary tests; browser, performance and live-network tests were excluded. Line coverage over apps/ was 93 %.

6. Proposed Research Extension

This section describes experiments that have not been run. It adds no new system: the existing application, data generator, indexes and transaction helper are reused, and ArangoDB remains the only database.

6.1 Evidence status

Table 11: Distinction between implemented, recorded and proposed work

Status	Content
Completed implementation	Revision-aware ingestion, 31 collections, 32 secondary indexes, graph, ArangoSearch view, schema validation, stream transactions with conflict retry, CRUD API, 15 AQL queries.
Previously recorded evidence	Single-client, sequential index benchmark (docs/BENCHMARKS.md, 25 September 2026) comparing five read workloads with and without the read indexes at 500, 10,000 and 100,000 events; integrity indexes stayed in place. For example, at 100,000 events revision lookup p95 fell from 72.5 ms to 3.6 ms, while at 500 events the indexes gave no benefit. This evidence contains no concurrent writers.
Proposed experiments	Concurrent read/write workloads over three index configurations, measuring latency, throughput, resource use, conflicts and revision consistency (this section).

6.2 Index configurations

Table 12: Compared ArangoDB index configurations

Configuration	Definition
B0 - Minimal	Mandatory integrity indexes only: alert_matches.qw_matches_rule_event, auth_users.qw_auth_username, event_revisions.qw_revisions_event_rev, events.qw_events_source_identity, jobs.qw_jobs_dedupe.
B1 - Current	All 32 existing QuakeWatch secondary indexes (Table 5).
B2 - Tuned	Index set chosen after analysing the B0 and B1 measurements (for example removing an index that costs write throughput without helping a measured read, or adding a compound index for a measured slow read). It is defined only after the data exist.

6.3 Workloads and datasets

Table 13: Concurrent workload distributions

Workload	Read share	Write share	Operations
Read-heavy	80 %	20 %	Reads: filtered event list, radius search, revision lookup. Writes: transactional event upsert that appends a revision.
Balanced	50 %	50 %	Same operations, different mix.
Write-heavy	20 %	80 %	Same operations, different mix.

Datasets are synthetic, schema-valid earthquake catalogues produced by the existing benchmark generator in sizes of 10,000 and 50,000 events, with 100,000 events as an optional third size. Every configuration is loaded from the same seed, so datasets and operation sequences are identical across B0, B1 and B2, and the uniqueness constraints are never removed.

6.4 Metrics and consistency check

Table 14: Proposed metrics

Metric	Proposed measurement
p50 / p95 latency	Client-observed time per read and per write operation
Read / write throughput	Completed operations per second per operation type
CPU and memory	Resource use of the ArangoDB container during each run
Storage overhead	Collection and index sizes reported by ArangoDB after loading
Conflicts and retries	Write-write conflicts (error 1200) and transaction retries
Revision consistency	After each run: every event's revision_count equals its number of revision documents; revision numbers are contiguous from 1; no duplicate (event, revision) pair; no revision without an event

6.5 Controls and limits

- Warm-up runs precede timed runs, and each configuration and workload is repeated; medians and spread are reported.
- All runs use the same single-node Docker deployment; results will describe that environment only.
- The objective is to describe workload-specific trade-offs within ArangoDB. No comparison with other databases and no claim of improvement is made before measurements exist.

7. Conclusion and Future Work

QuakeWatch demonstrates a working revision-aware earthquake data store on ArangoDB: 2,670 live events, 31 collections, 32 secondary indexes, a search view and a graph; server-side schema validation, unique constraints and transactions that stand in for relational triggers and procedures; fifteen executed AQL queries with screenshot evidence; a CRUD flow whose status codes were verified against stored data; and an executed test set in which every initial failure was repaired or explained and retested. The index benchmark recorded earlier is single-client and read-only, which leaves open how indexes affect concurrent writes and revision consistency. Future work is therefore the experiment of Section 6: running B0, B1 and B2 under the three workload mixes at 10,000 and 50,000 events (and optionally 100,000), deriving B2 from the measurements, and reporting the results together with their limits.

- **Live data.** Counts change every five minutes; tables report the values observed during the runs.

- **Backup scope.** The arangodump artifact came from an isolated test database; no restore test was carried out.
- **Repository state.** The sign-up repair and its test are in the working tree and the deployed image; they must be committed and pushed to appear on GitHub.
- **Screenshots.** Figures were captured from the ArangoDB web interface; textual results were produced independently by scripts.

References

[1] ArangoDB Inc., "ArangoDB documentation, version 3.12: AQL, indexing, ArangoSearch, graphs and transactions," 2026. [Online]. Available: https://docs.arangodb.com/3.12/ (accessed Oct. 11, 2026).

[2] U.S. Geological Survey, Earthquake Hazards Program, "GeoJSON summary and detail feeds," 2026. [Online]. Available: https://earthquake.usgs.gov/earthquakes/feed/v1.0/geojson.php (accessed Oct. 11, 2026).

[3] Django Software Foundation, "Django documentation: Authentication, sessions and CSRF protection," 2026. [Online]. Available: https://docs.djangoproject.com/ (accessed Oct. 11, 2026).

[4] Encode OSS Ltd., "Django REST framework: Authentication and throttling," 2026. [Online]. Available: https://www.django-rest-framework.org/ (accessed Oct. 11, 2026).

[5] Postman, Inc., "Postman Learning Center: Collections, scripts and the Newman command-line runner," 2026. [Online]. Available: https://learning.postman.com/ (accessed Oct. 11, 2026).

[6] A. B. Reddy, "QuakeWatch: index benchmarks (docs/BENCHMARKS.md)," generated Sep. 25, 2026. [Online]. Available: https://github.com/abharathreddy751/quake_watch_v2.git